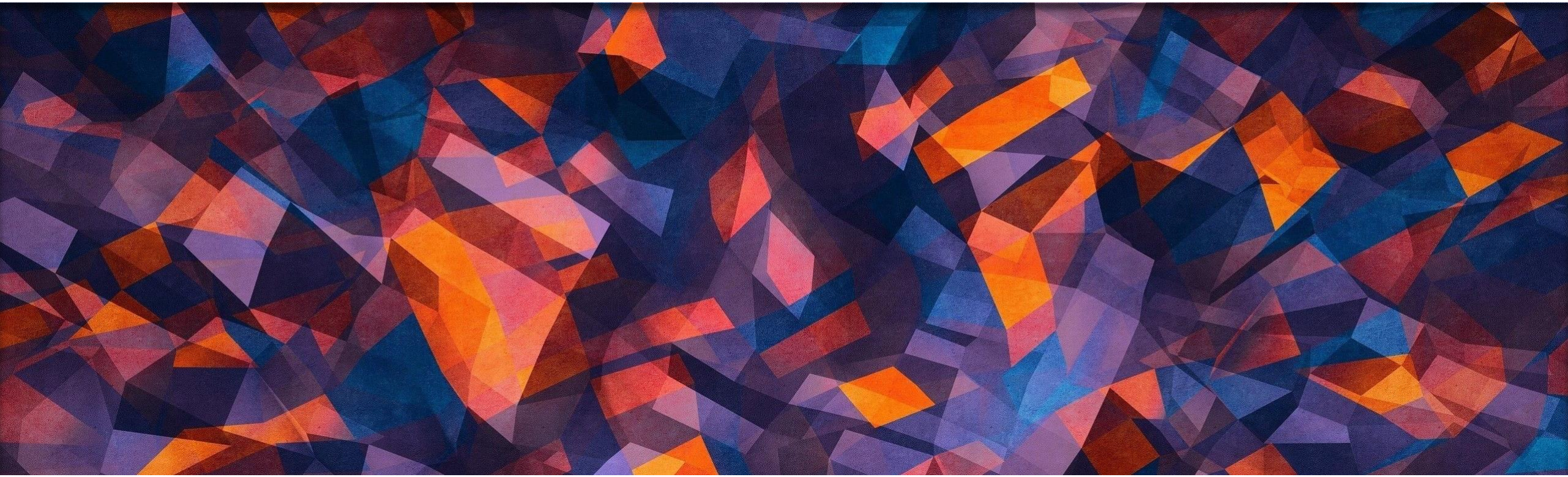
Universität St.Gallen



Event-driven and Process-oriented Architectures

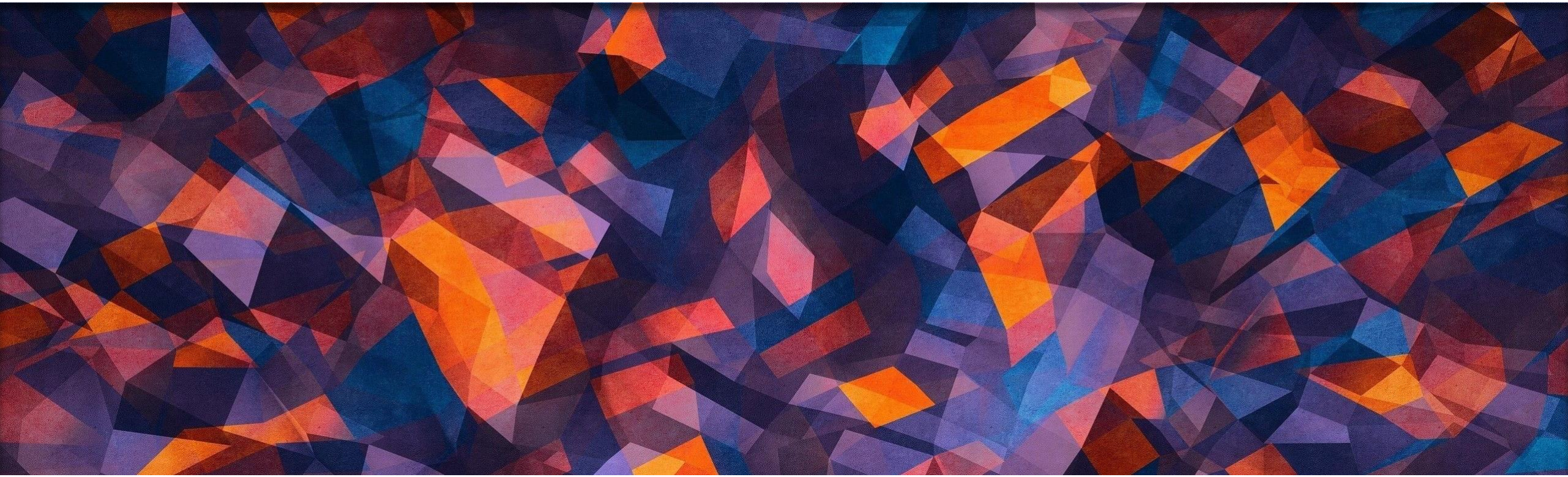
Integration Challenges and Transactional Sagas

Lecture 5

Agenda

- Transactional Sagas
- Process Engines and Integration Challenges

Universität St.Gallen



Event-driven and Process-oriented Architectures

Transactional Sagas

Lecture 5

Saga Pattern Variants

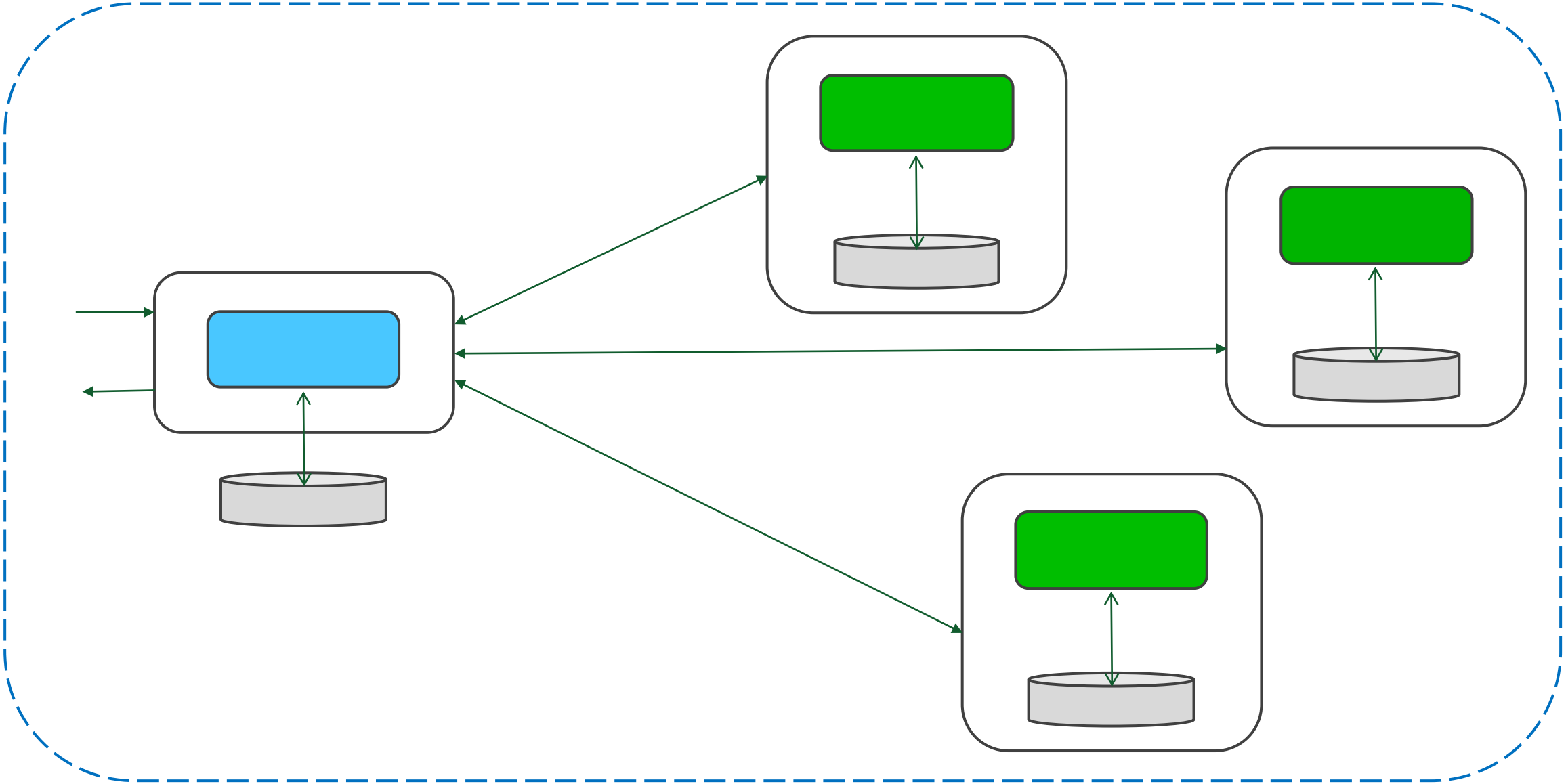
Pattern Name	Communication	Consistency	Coordination	Coupling
Epic Saga	Synchronous	Atomic	Orchestrated	Very high
Phone Tag Saga	Synchronous	Atomic	Choreographed	High
Fairy Tale Saga	Synchronous	Eventual	Orchestrated	High
Time Travel Saga	Synchronous	Eventual	Choreographed	Medium
Fantasy Fiction Saga	Asynchronous	Atomic	Orchestrated	High
Horror Saga	Asynchronous	Atomic	Choreographed	Medium
Parallel Saga	Asynchronous	Eventual	Orchestrated	Low
Anthology Saga	Asynchronous	Eventual	Choreographed	Very low

Focus of this lecture: Epic → Fairy Tale → Anthology Saga

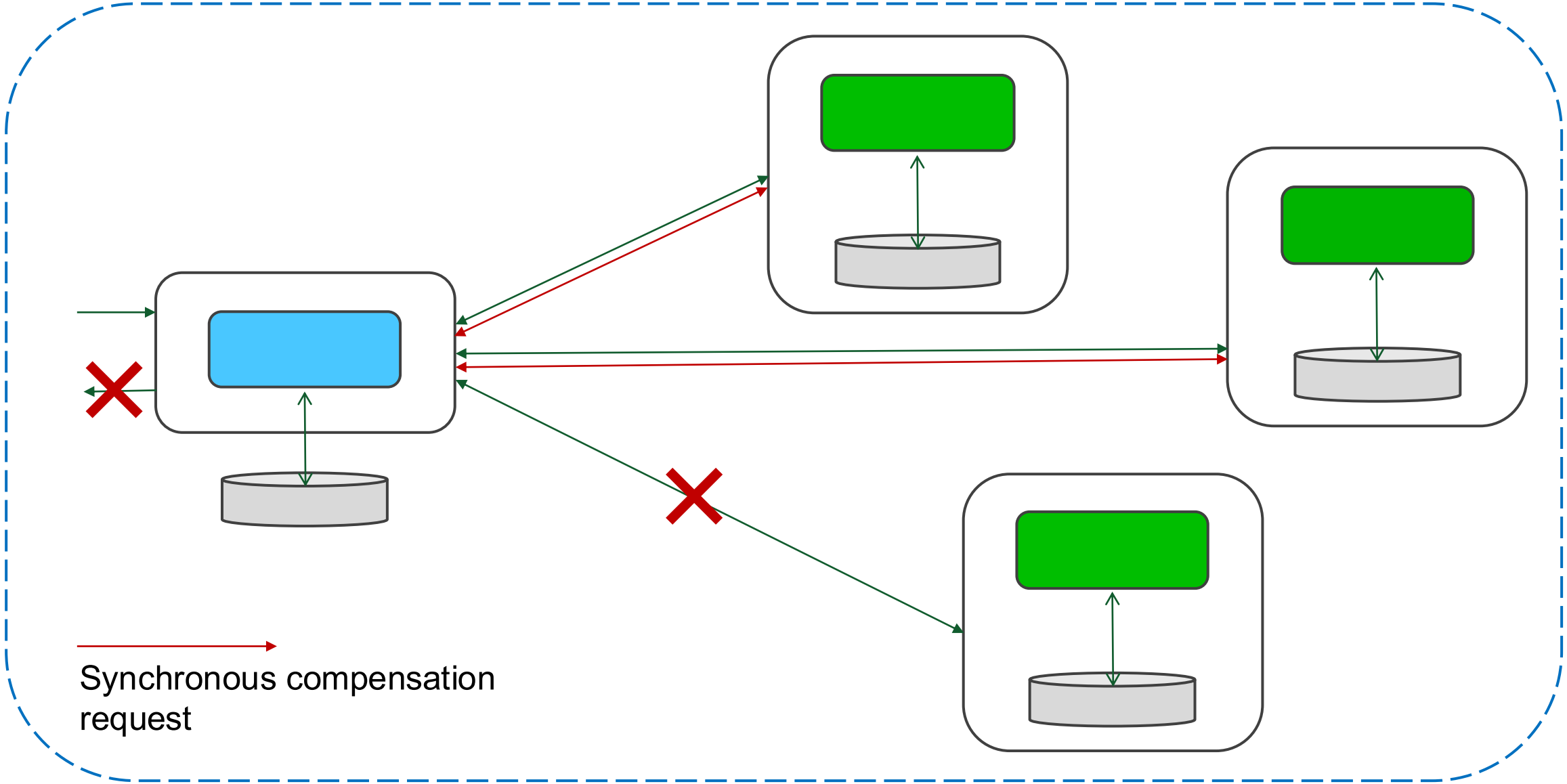
Epic Saga Pattern (aka Orchestrated Saga)

- The Epic Saga pattern refers to the traditional Saga Pattern
- Transactional coordination mimics monolithic systems
- Has orchestrator component monitoring the transactional completeness of a request
- Compensating transactions to reverse data write action performed by another service during the course of a distributed transaction scope

Epic Saga



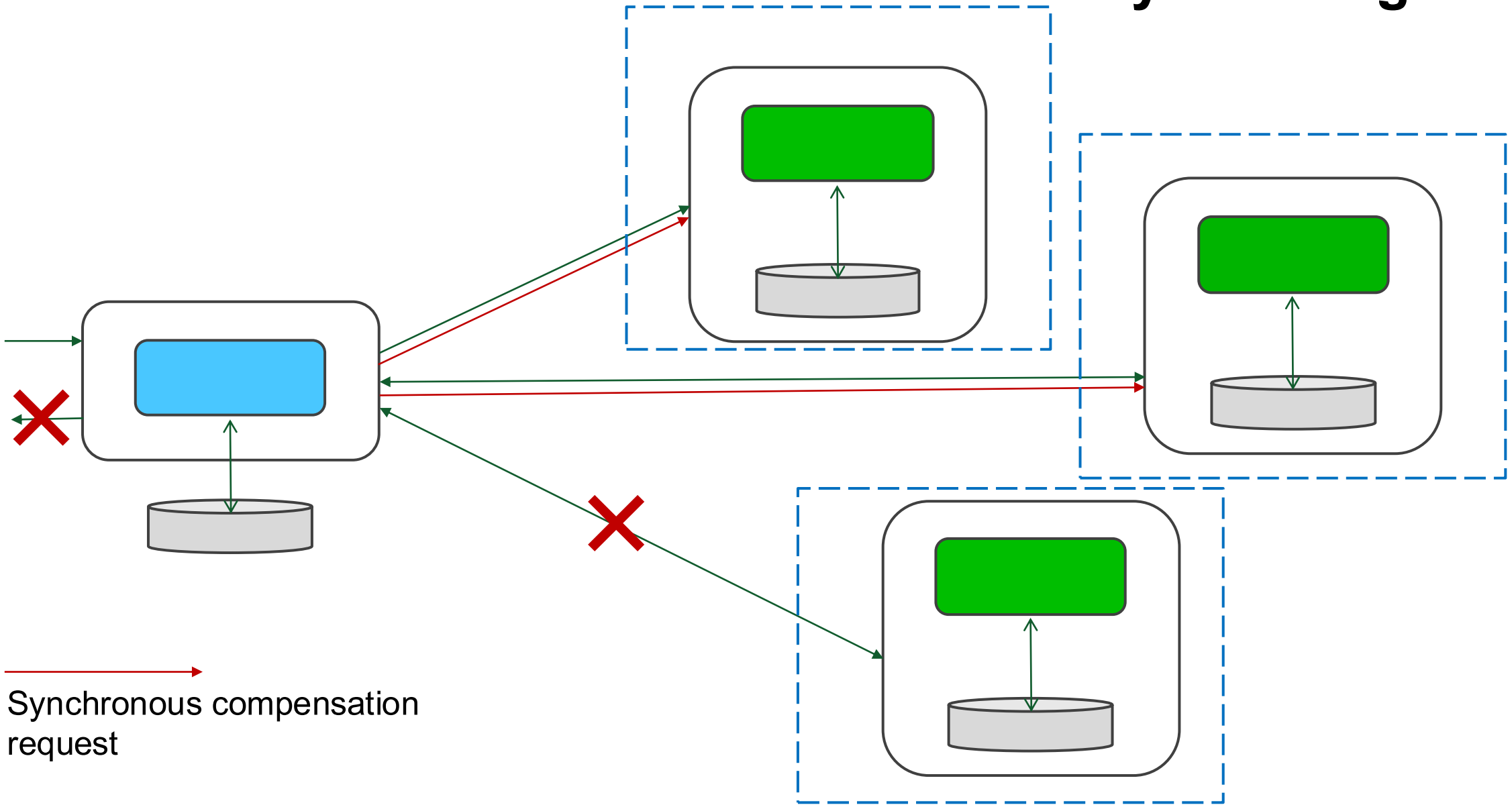
Epic Saga



Fairy Tale Saga Pattern

- The Fairy Tale Saga pattern relaxes the difficult atomicity requirement
- Has orchestrator component to coordinate request, response and error handling, but without the responsibility for managing transactions
- Orchestrator can manage compensating actions without the requirement of occurring within an active transaction
- Domain services manage their own transactional behavior, relying on eventual consistency

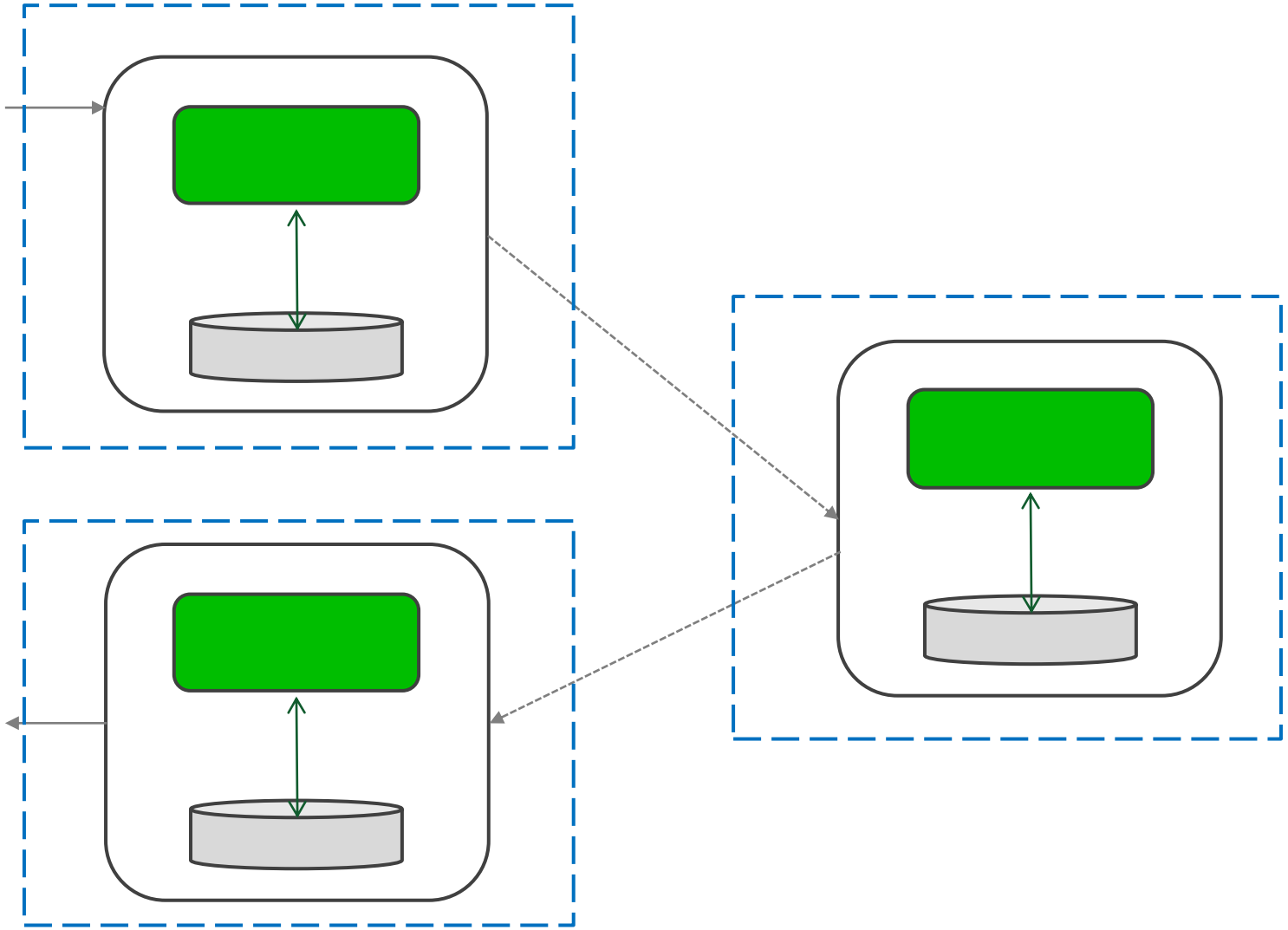
Fairy Tale Saga



Anthology Saga

- Provides the exact opposite set of characteristics to the traditional Epic Saga pattern
- Lacks a central orchestrator
- Each service maintains each own transactional integrity; services required to include context about the workflows they participate (including error handling and other coordination strategies)
- Stamp coupling might be used together with this pattern to carry workflow state

Anthology Saga



Trade-offs Overview

Pattern Name	Communication	Consistency	Coordination	Coupling
Epic Saga	Synchronous	Atomic	Orchestrated	Very high
Fairy Tale Saga	Synchronous	Eventual	Orchestrated	High
Anthology Saga	Asynchronous	Eventual	Choreographed	Very low

Focus of this lecture: Epic → Fairy Tale → Anthology Saga

Epic Saga

Attribute	Rating
Communication	Synchronous
Consistency	Atomic
Coordination	Orchestrated
Coupling	Very high
Complexity	Low
Responsiveness/Availability	Low
Scalability/Elacticity	Very low

Fairy Tale Saga

Attribute	Rating
Communication	Synchronous
Consistency	Eventual
Coordination	Orchestrated
Coupling	High
Complexity	Very low
Responsiveness/Availability	Medium
Scalability/Elacticity	High

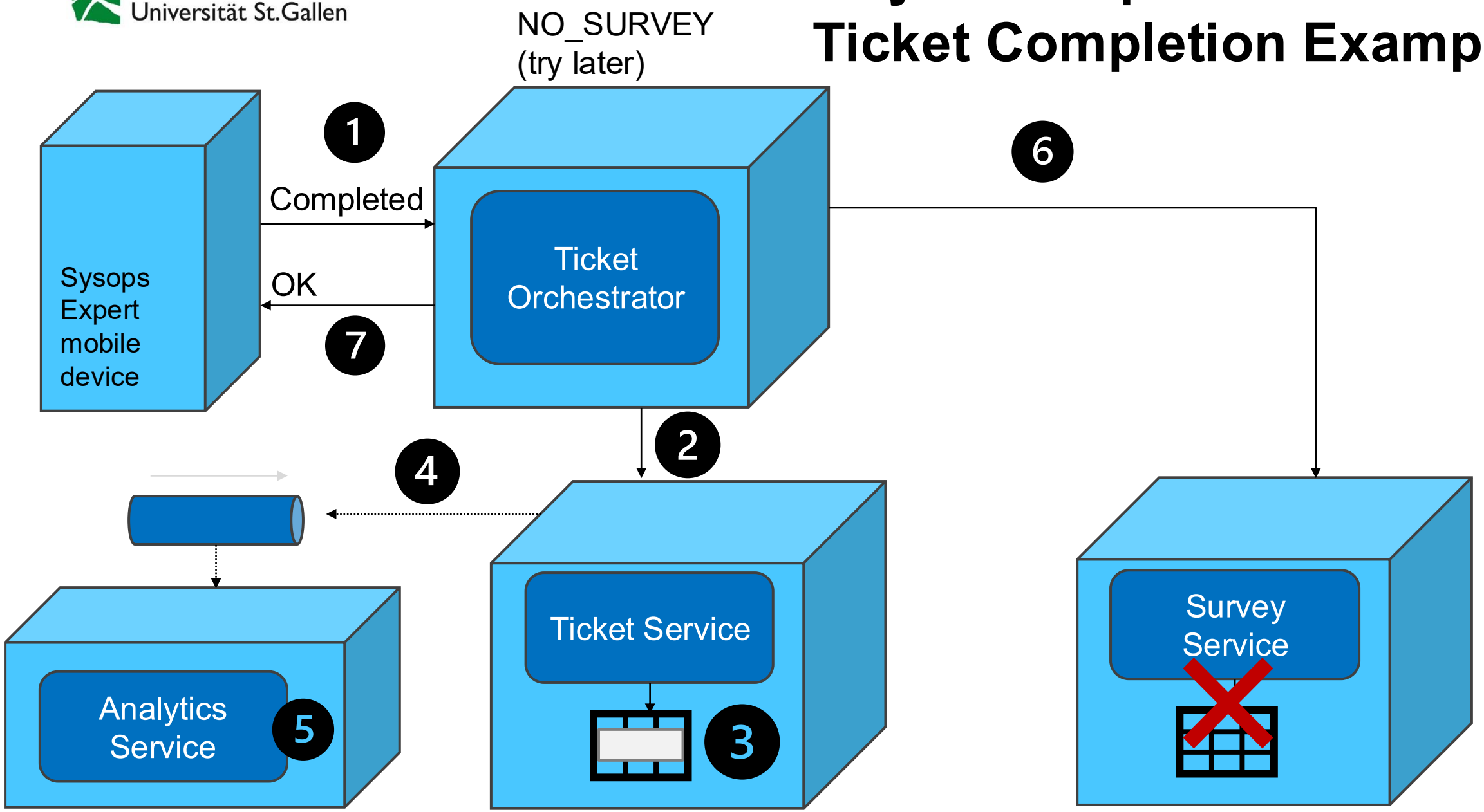
Anthology Saga

Attribute	Rating
Communication	Asynchronous
Consistency	Eventual
Coordination	Choreographed
Coupling	Very low
Complexity	High
Responsiveness/Availability	High
Scalability/Elacticity	Very high

Managing Eventual Consistency with Workflow State

- A workflow engine or finite state machine explicitly tracks the current state of a long-running saga
- Error conditions can be corrected through retries or some sort of automated or manual corrective action
- Instead of undoing actions with compensating updates, the workflow tracks the current state and continues the process until a consistent state is reached.

Fairy Tale Implementation of Ticket Completion Example



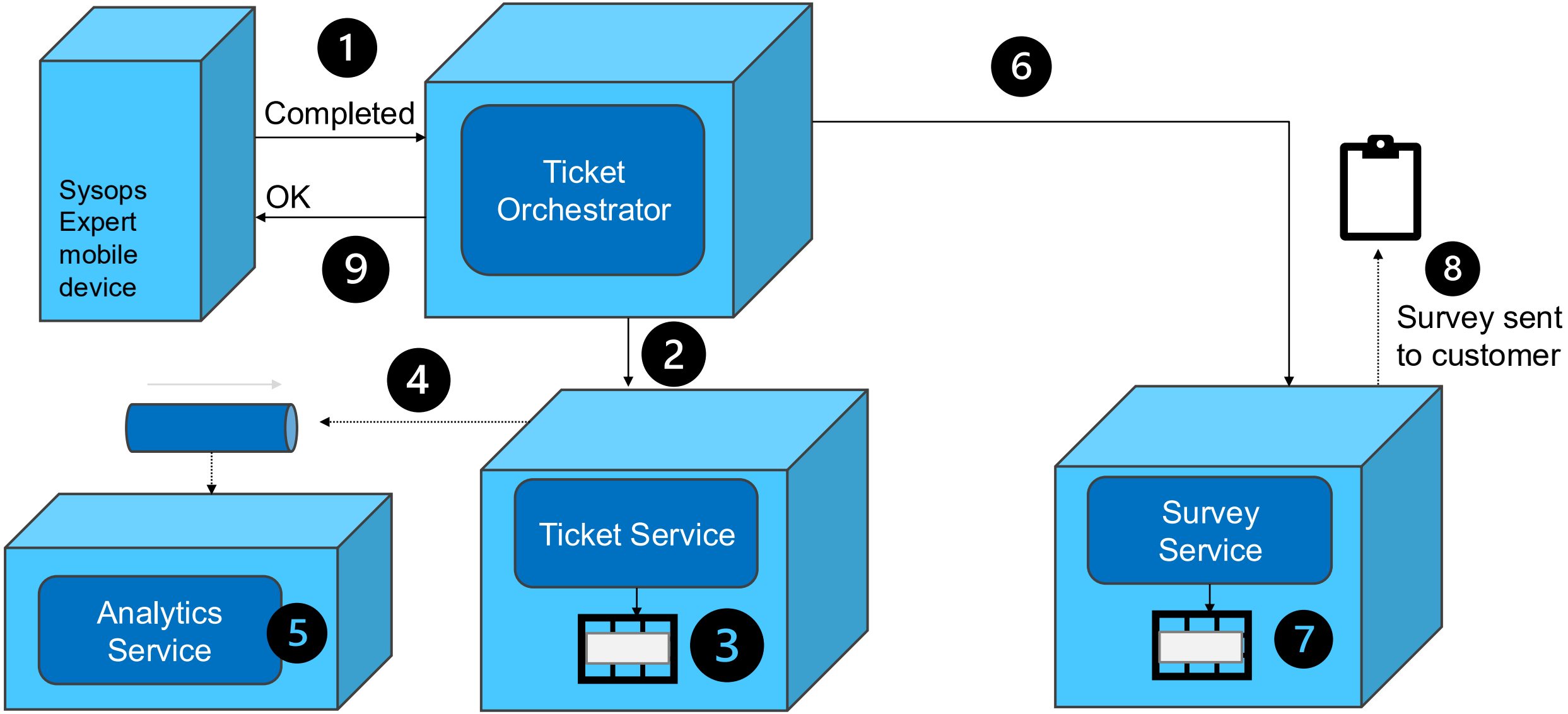
Trade-offs of State Management
(rather than atomic transactions with compensating updates)

Advantages	Disadvantages
Good responsiveness	Data might be out of sync when errors occur
Less impact to end-user for errors	Eventual consistency might take some time

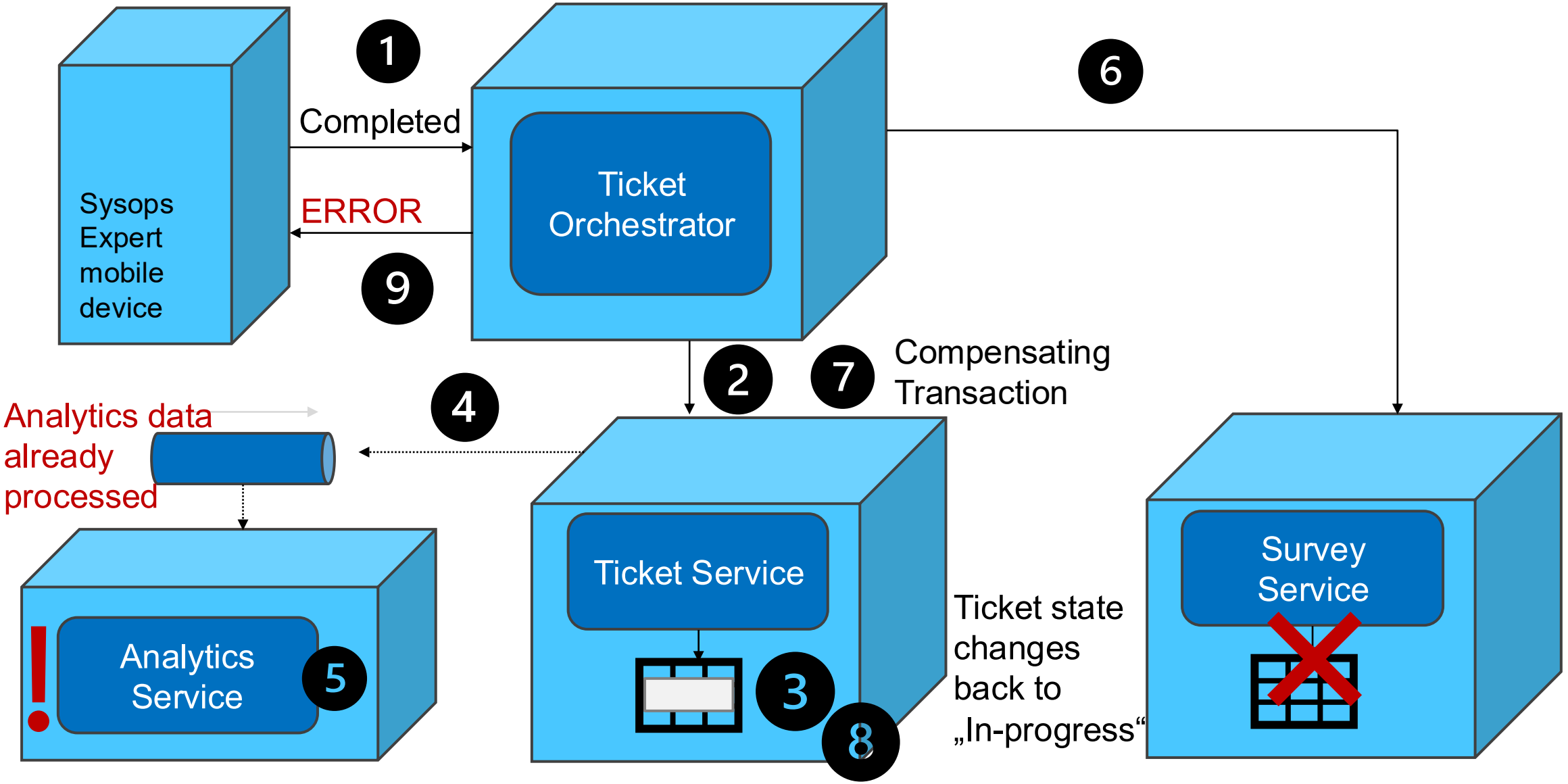
Atomic Transactions and Compensating Updates

- Instead of managing workflow state with eventual consistency, architects may attempt to maintain atomic behavior using transactions and compensating updates.

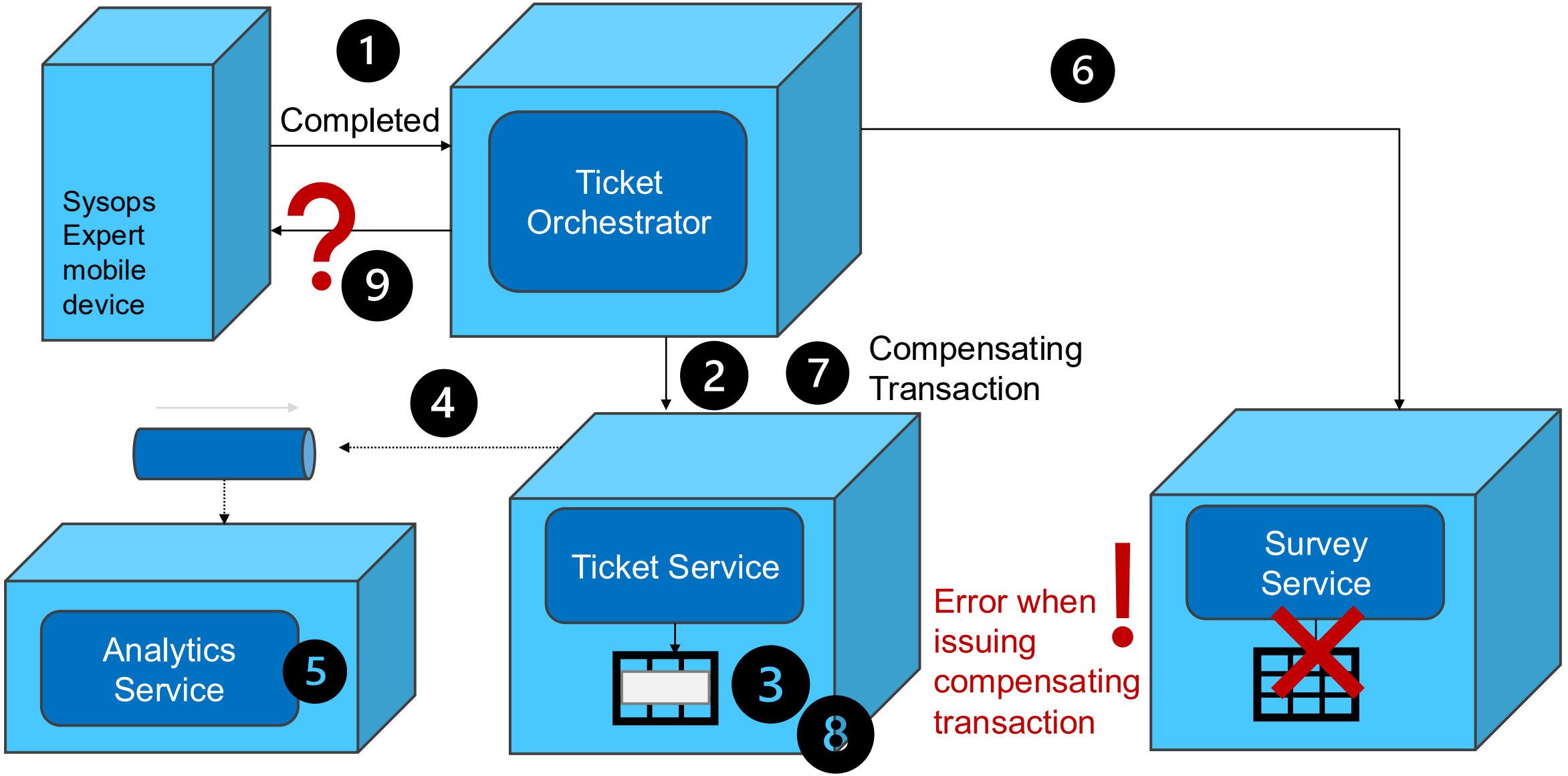
Epic Saga Implementation of Ticket Completion Example



Epic Saga Requires Compensation but Side Effects May Occur



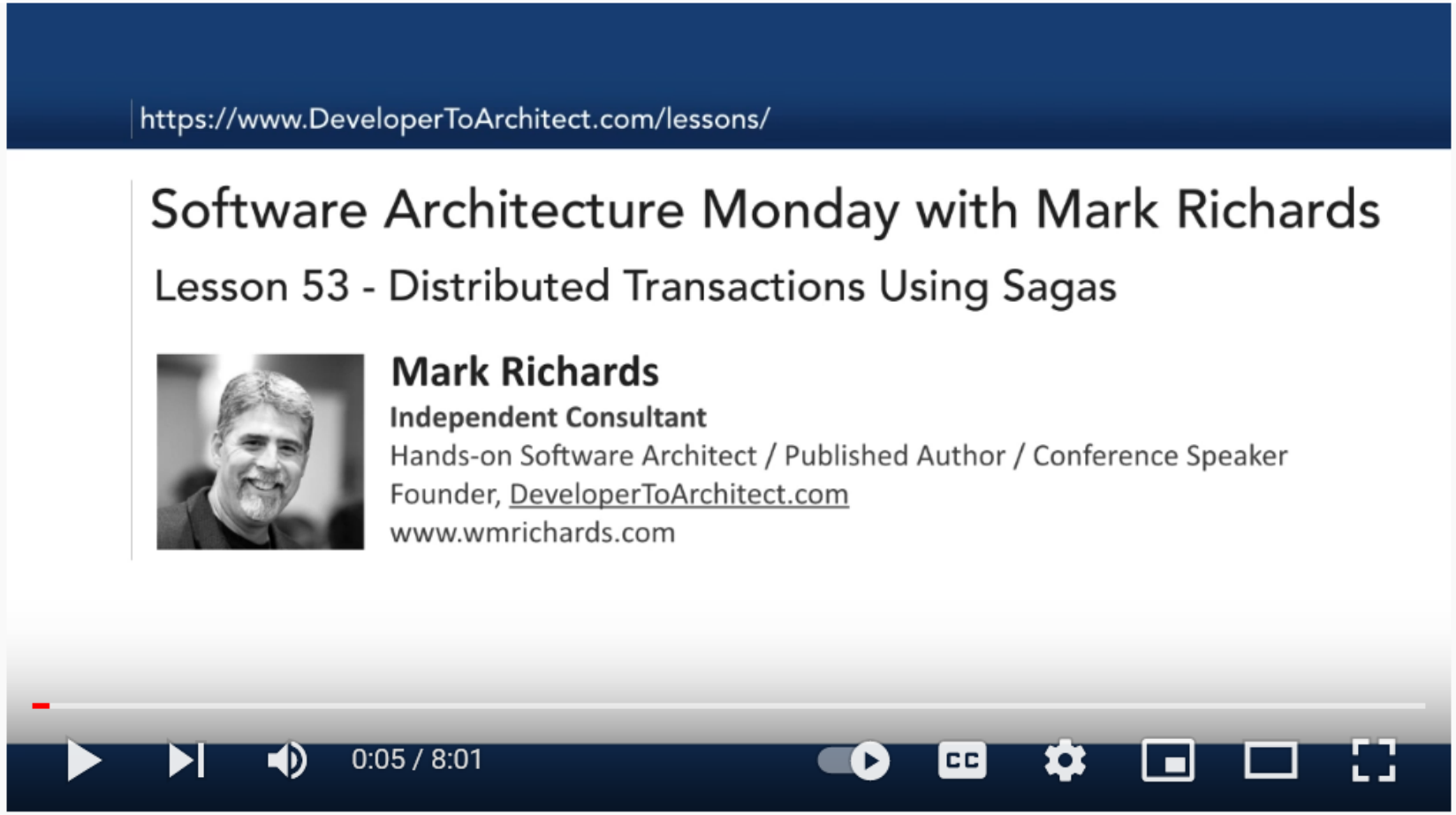
Compensating Updates within an Epic Saga Can Fail



Trade-offs with Atomic Transactions with Compensating Updates

Advantages	Disadvantages
All data restored to prior state	No true transaction isolation across services
Allows retries and restart	Side effects may occur on compensation
Strong control over the process outcome	Compensation may fail
	Poor responsiveness for the end user

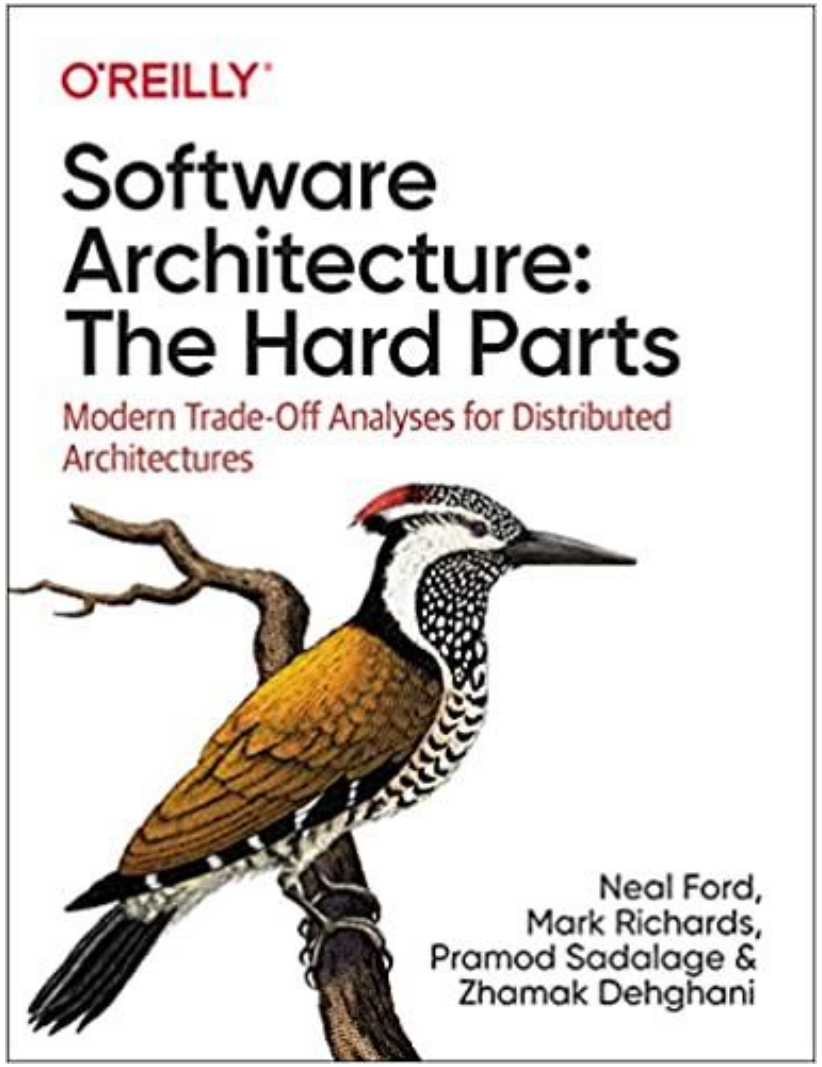
Distributed Transactions Using Sagas



Mark Richards advocates the usage of state machines (similar to workflow engines) for managing state and the usage of annotations

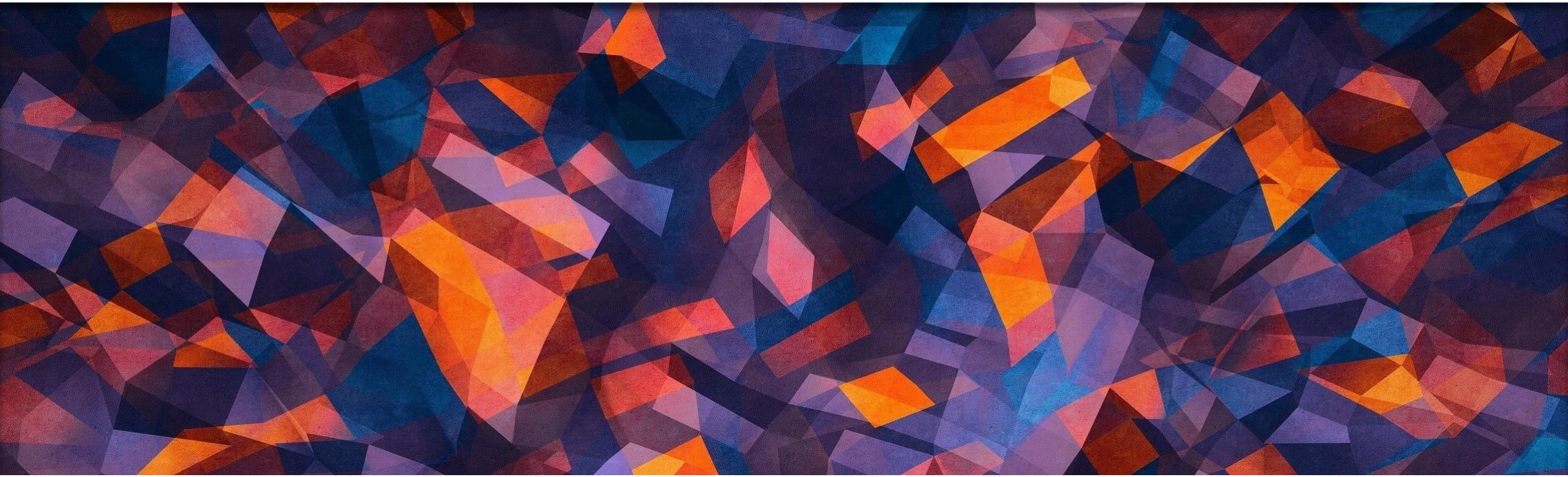
Universität St.Gallen

References and Further Material



Based on Chapter 12 – Transactional Sagas

Universität St.Gallen



Event-driven and Process-oriented Architectures

Process Engines and Integration Challenges

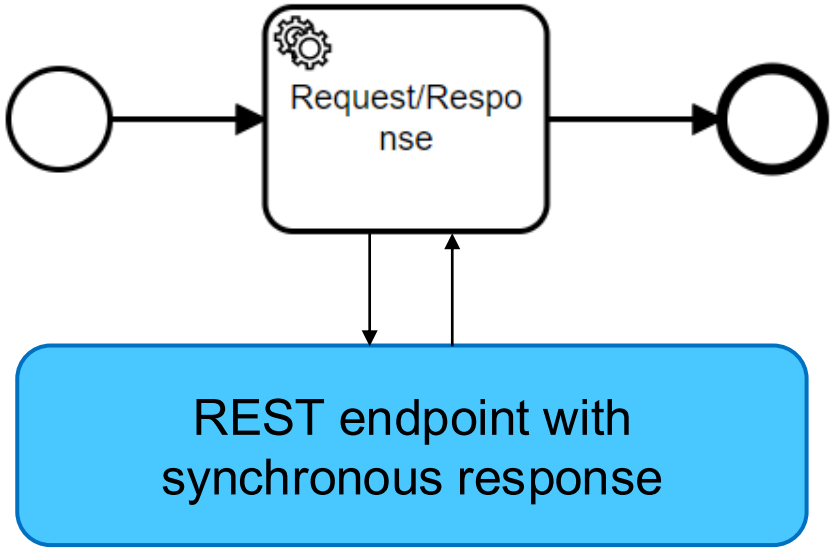
Lecture 5

Agenda

- **Communication patterns for service invocation, in particular, long-running and asynchronous communication**
- Consistency problems and transactional guarantees
- Importance of idempotence

Communication Patterns for Service Invocation
Synchronous Request/Response

- A REST call: a typical example of a synchronous request/response
- In BPMN a service task (as introduced in Week 3) is leveraged to invoke such a REST call



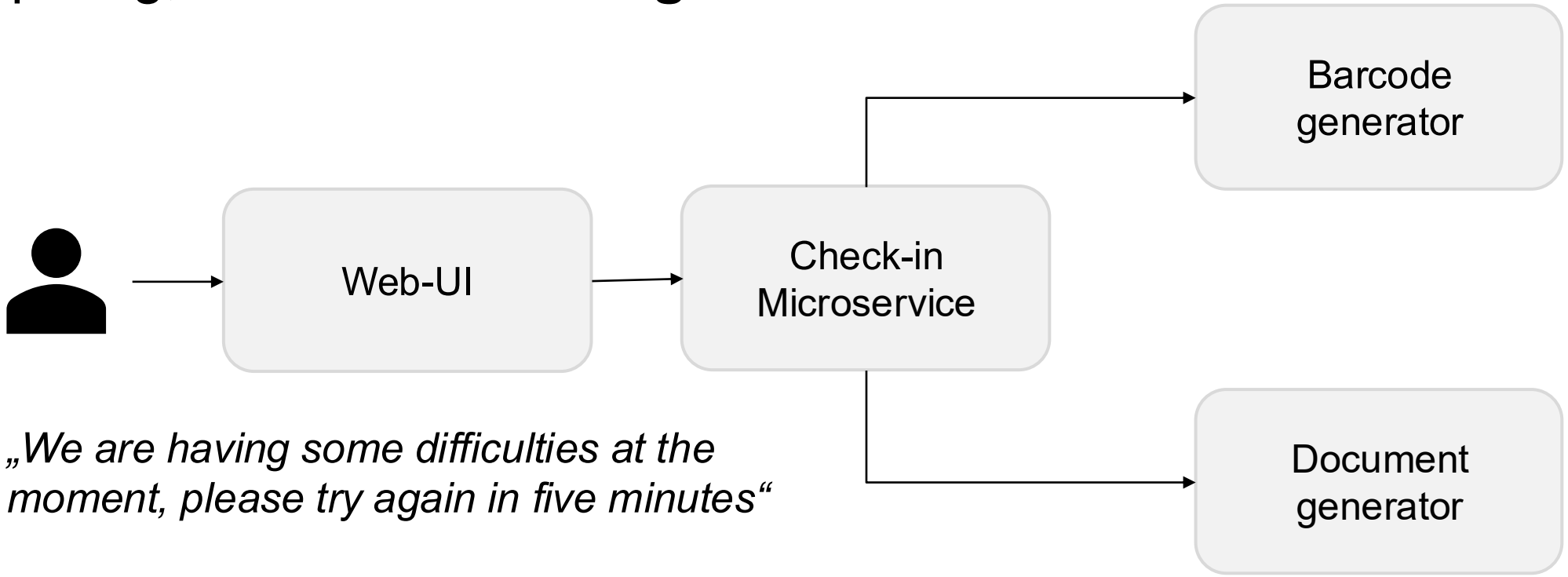
Even s simple REST call hides many distributed system challenges.

Remote communication is inherently unreliable as pointed out in the **fallacies of distributed computing**.

Often requires your services to be made **long-running**.

Communication Patterns for Service Invocation
Synchronous Request/Response

Since remote computing needs to respect the fallacies of distributed computing, it is crucial to **design for failure**.

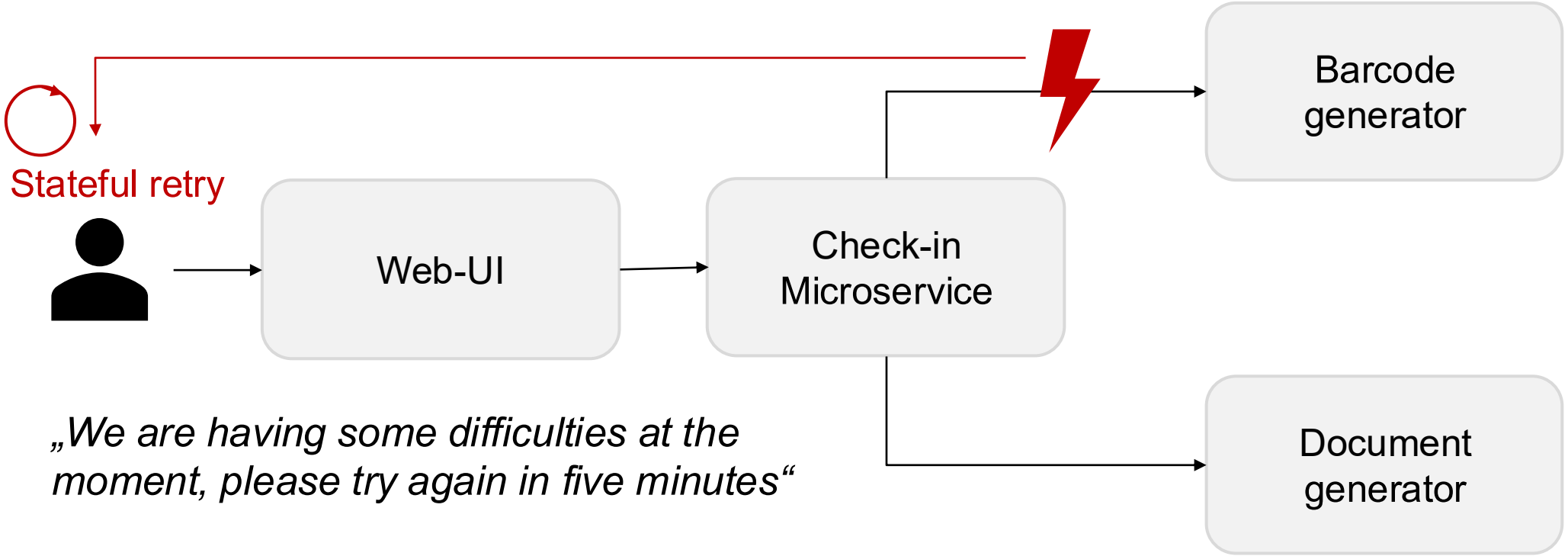


„We are having some difficulties at the moment, please try again in five minutes“

Fail fast patterns (e.g., Circuit Breaker) are the most common strategy to prevent local errors to bring down the whole system

Communication Patterns for Service Invocation
Synchronous Request/Response

Fail fast is not enough!

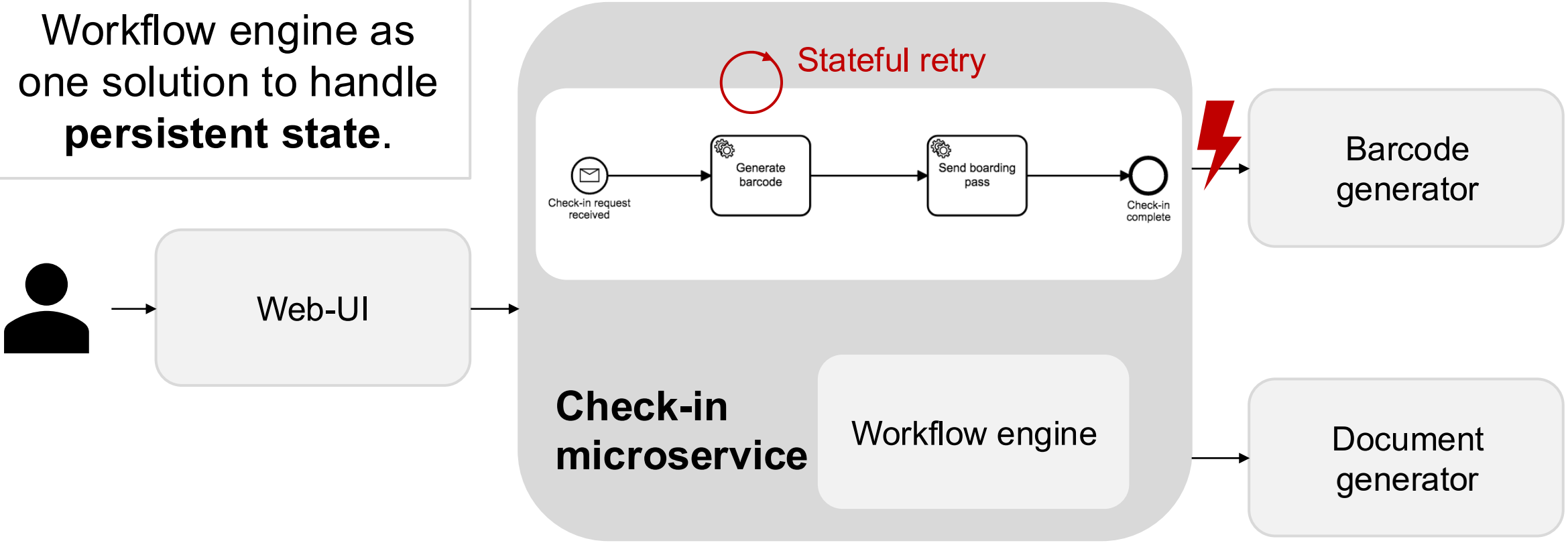


„We are having some difficulties at the moment, please try again in five minutes“

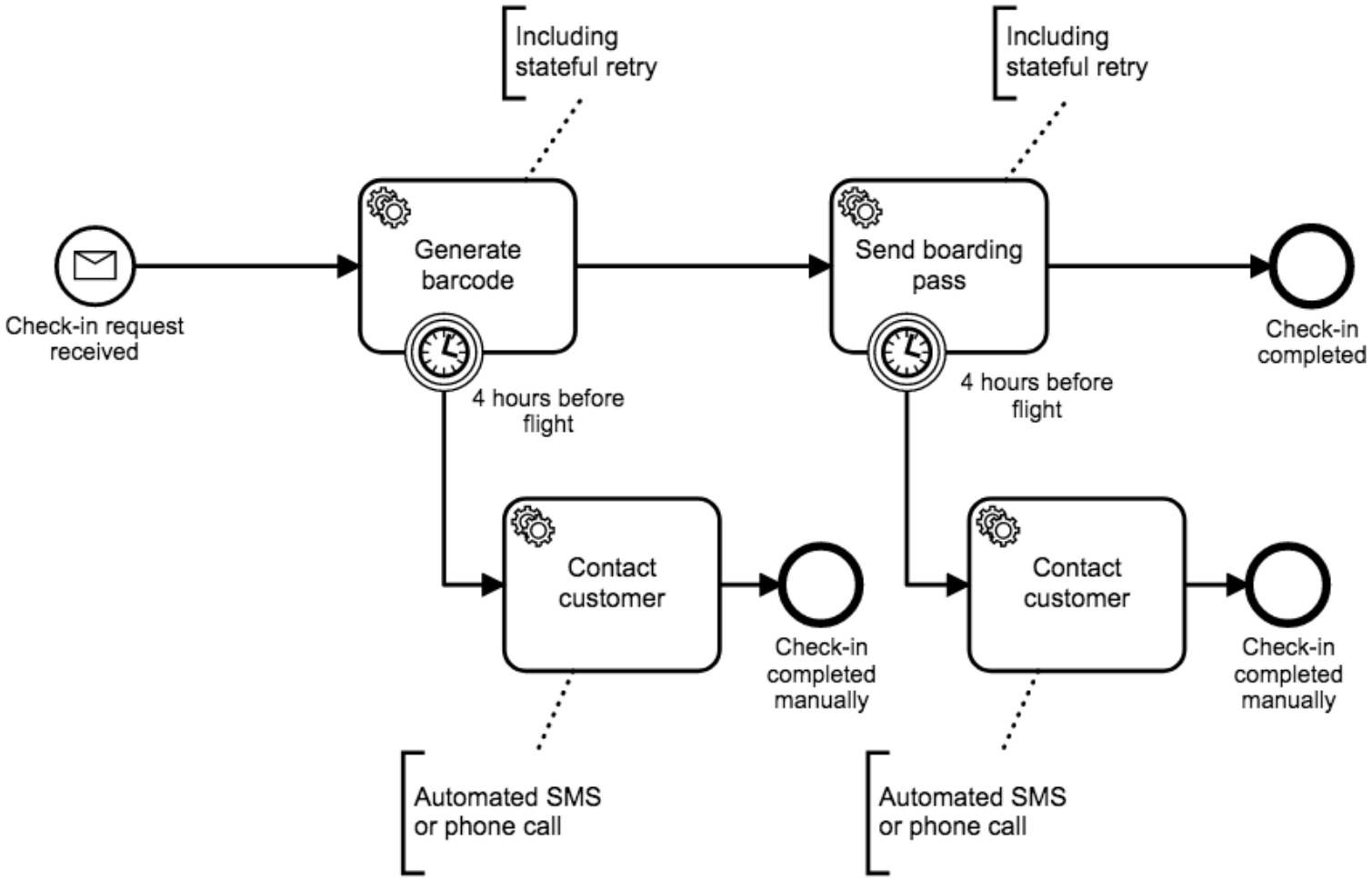
Without stateful retires, failures propagate back to the caller – often the user.

Stateful Resilience Pattern: Stateful Retry

Stateful retry should be kept **within the service** to keep the error local. Workflow engine as one solution to handle **persistent state**.



Handling Errors Responsibly

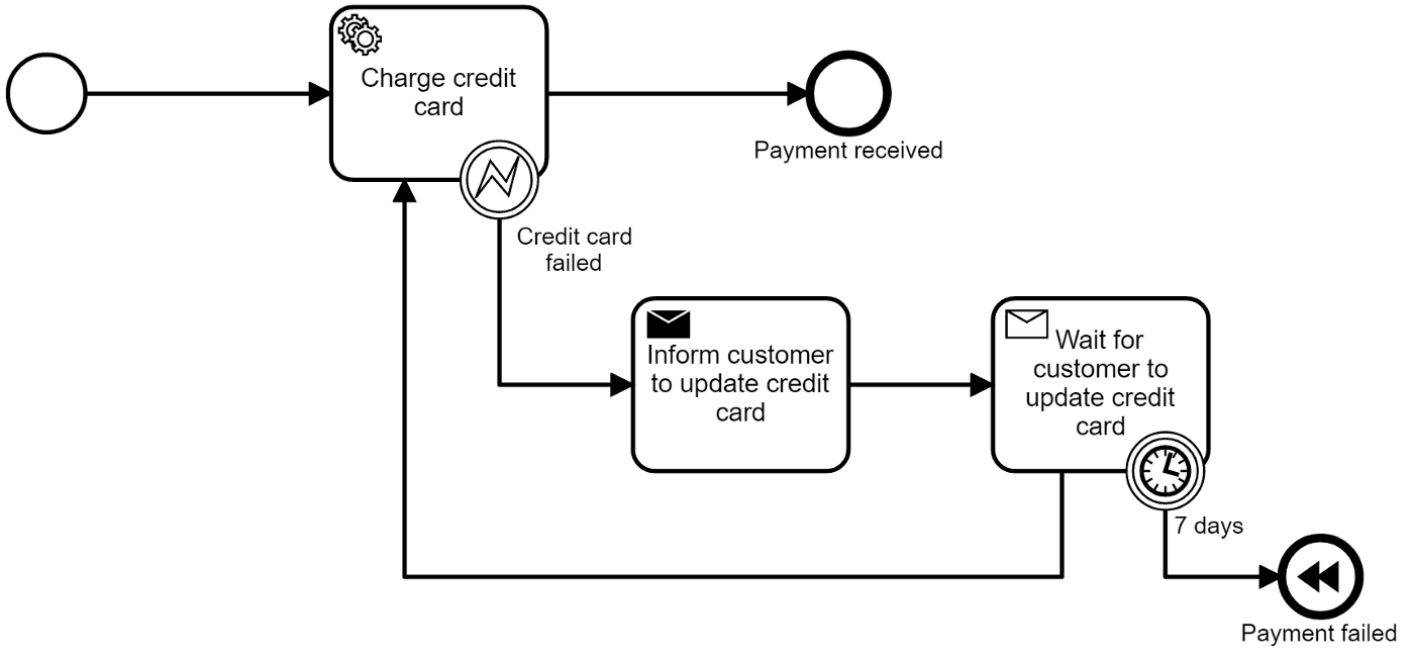


If the stateful retry is not successful, customers are contacted to check-in at the airport.

Similarly, if boarding passes cannot be printed the customer is contacted.

Source: https://camunda.com/blog/2018/08/stateful-orchestration-handle-errors-responsibly/

Stateful Resilience Pattern: Human Intervention

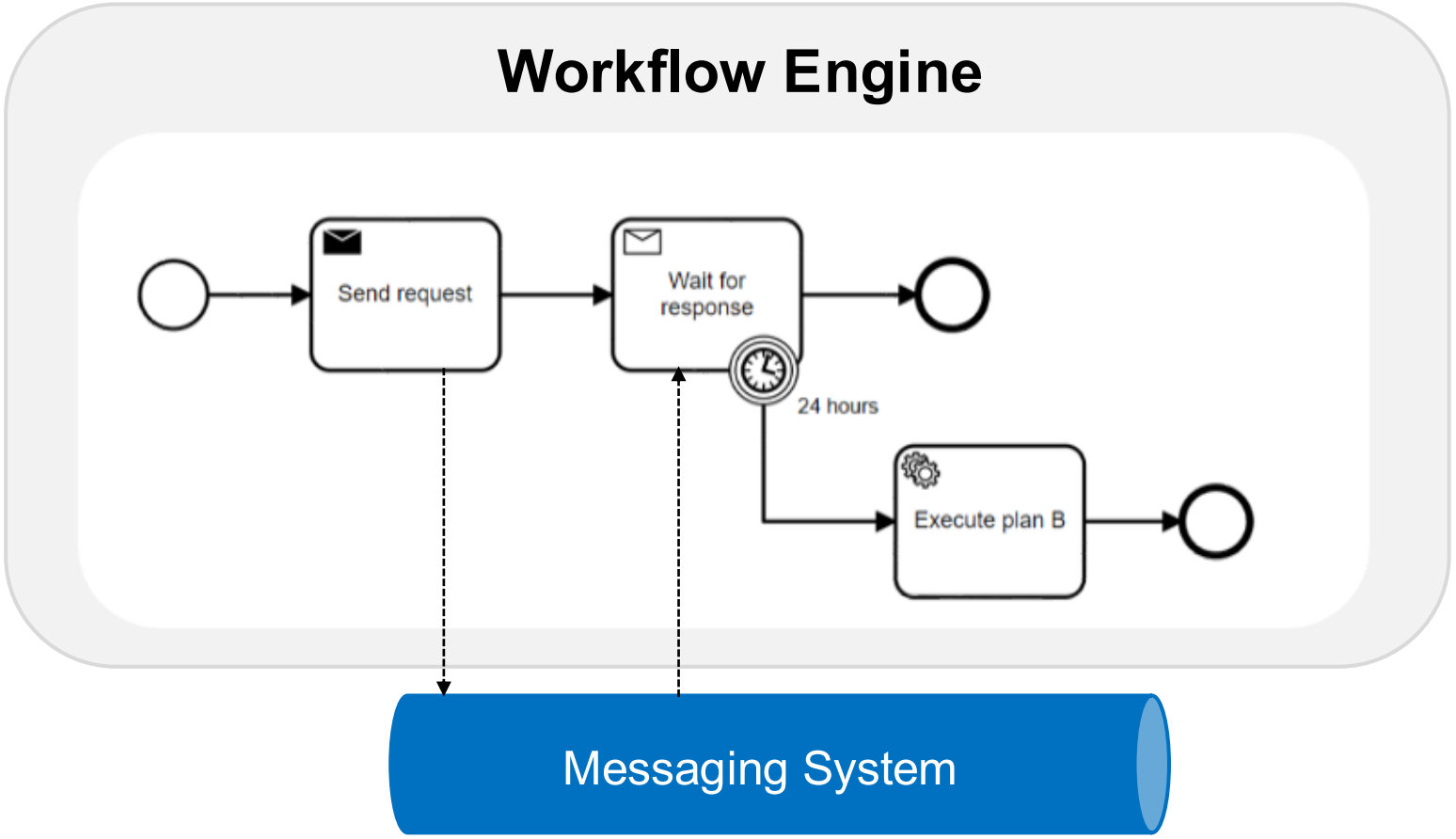


Human intervention is another stateful resilience pattern next to stateful retry.

Source: https://blog.bernd-ruecker.com/fail-fast-is-not-enough-84645d6864d3

Asynchronous Request / Response

Asynchronous communication is usually the domain of a messaging system.

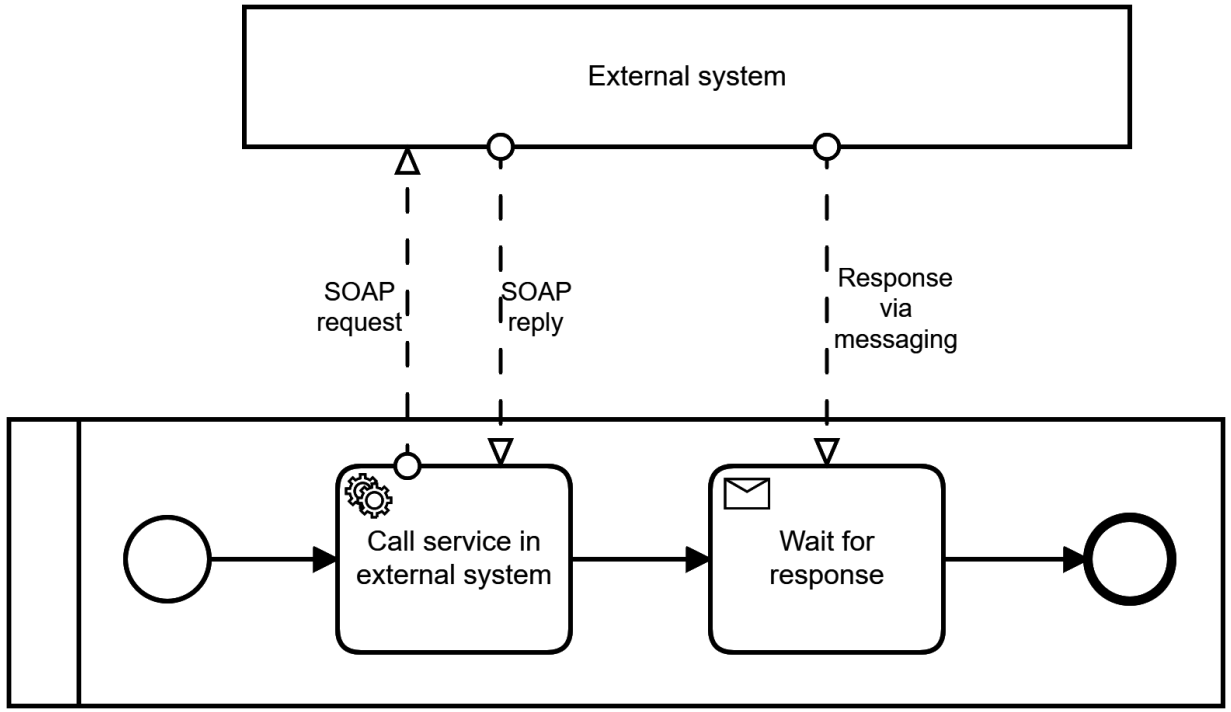


A workflow engine can help to handle the often **long-running** nature of communication

- **Handling of timeouts** to act on delays
- **Correlation mechanisms** to find the right waiting process instance

A Tricky Detail Concerning Timing

According to the BPMN standard a process instance needs to be ready to receive that specific message at the exact time the message arrives

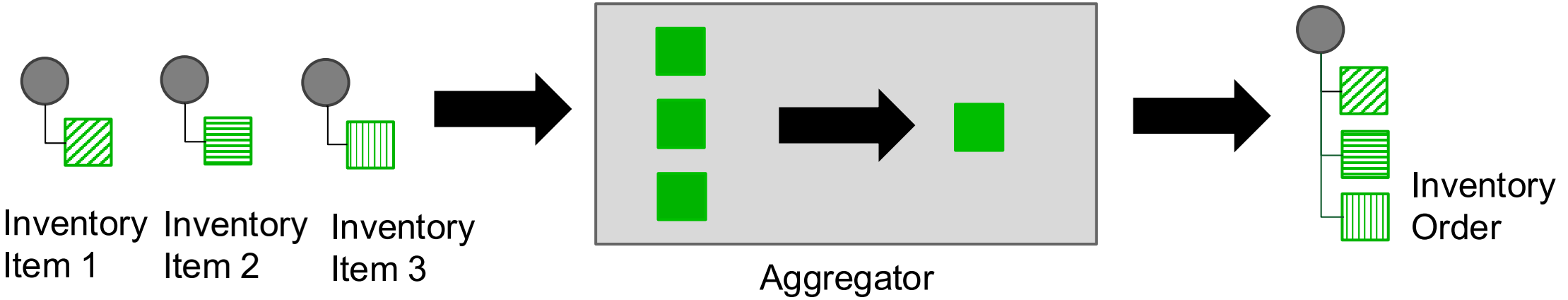


Camunda Cloud (but not Camunda Platform) provides **message buffering** to overcome this problem

Aggregating Messages: Aggregator Pattern

Collecting information accross multiple messages can be challenging in the context of a messaging system due to its asynchronous nature.

„Use a stateful filter, an Aggregator, to collect and store individual messages until a complete set of related messages has been received. Then, the Aggregator publishes a single message distilled from the individual messages.“

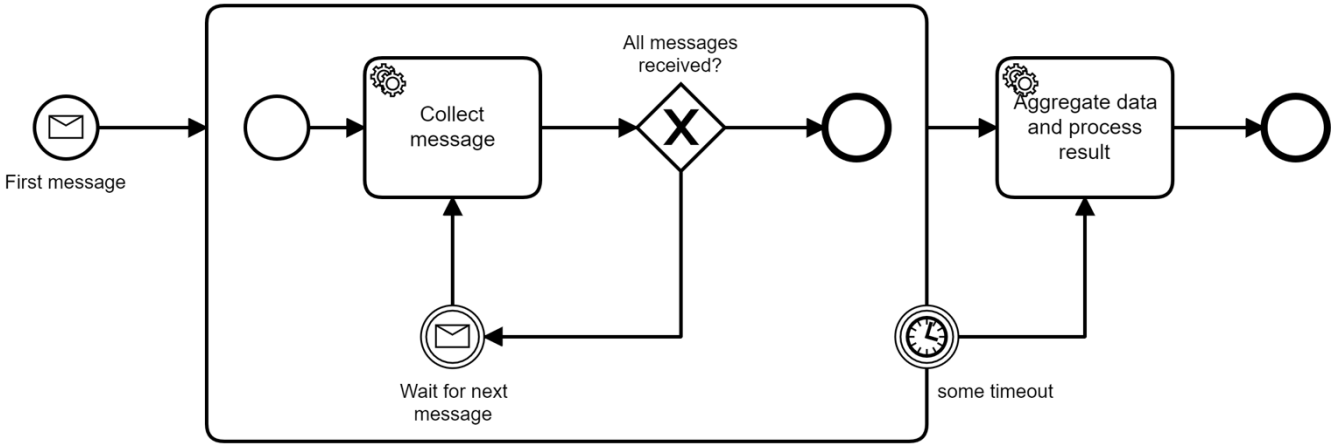


Source: Enterprise Integration Patterns

Aggregating Messages: Aggregator Pattern

Collecting information accross multiple messages can be challenging in the context of a messaging system due to its asynchronous nature.

„Use a stateful filter, an Aggregator, to collect and store individual messages until a complete set of related messages has been received. Then, the Aggregator publishes a single message distilled from the individual messages.“



Generic Aggregator implemented in BPMN

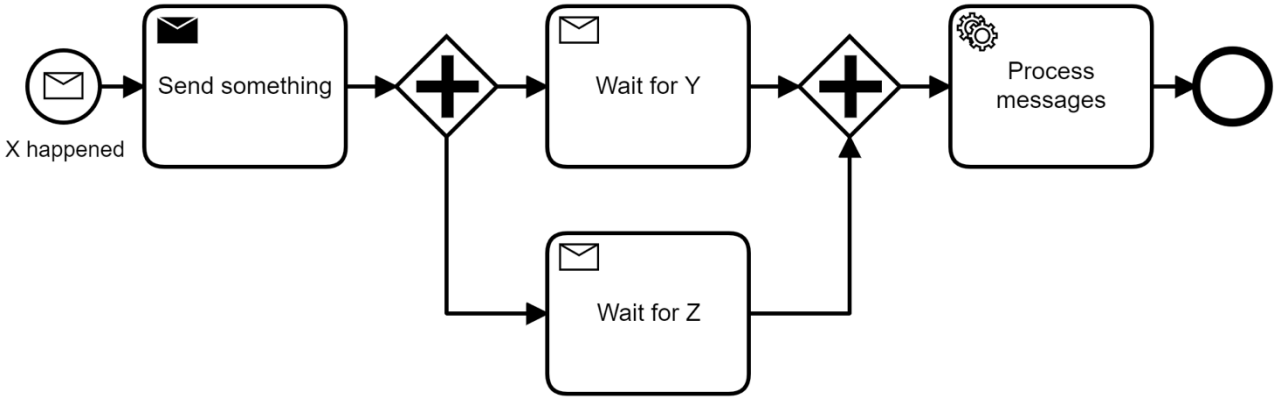
Using a workflow engine gives you **persistent state** and helps with **timeout handling**.
Message buffering might be needed to avoid that process instances are not ready to receive messages at the right time (or implement some workarounds).

Source: Enterprise Integration Patterns

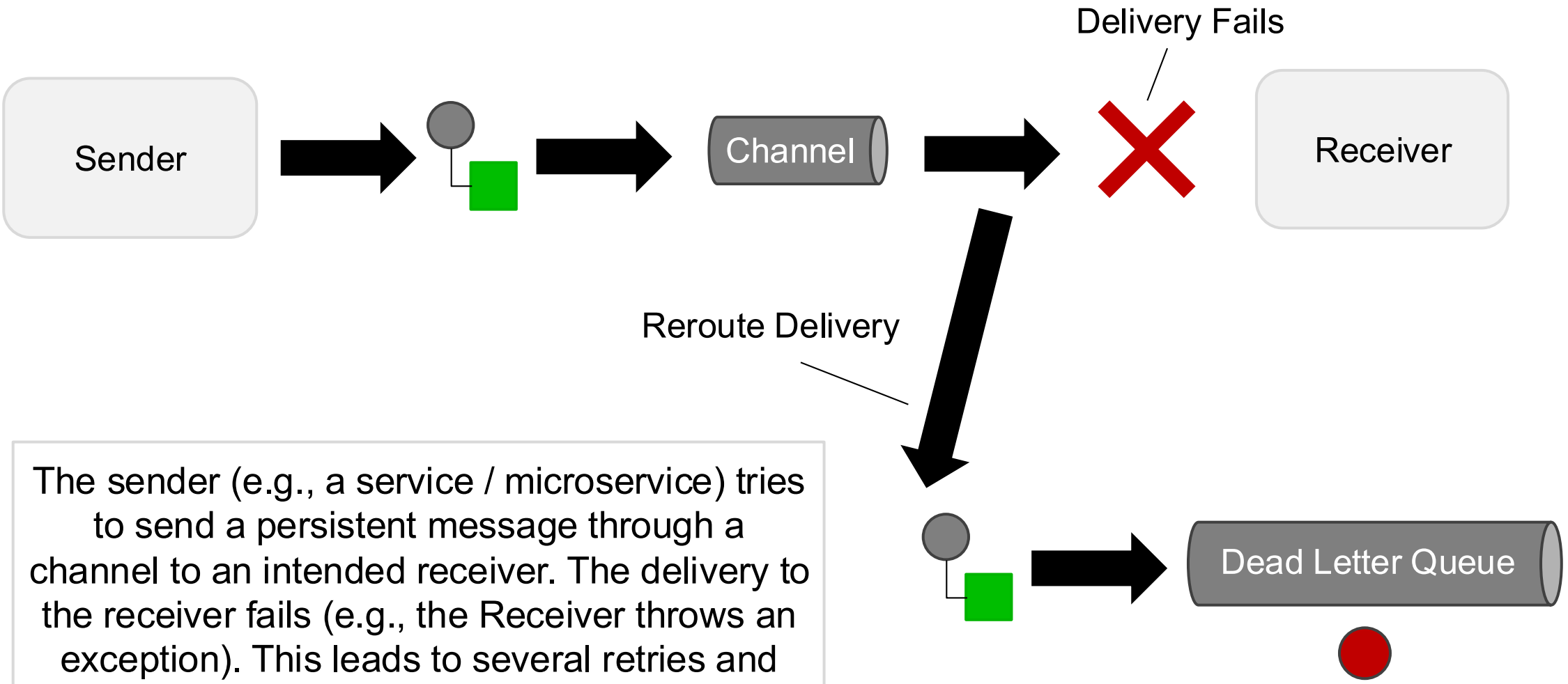
Aggregating Messages

Collecting information accross multiple messages can be challenging in the context of a messaging system due to its asynchronous nature.

Some business scenarios do not require the generic aggregator, but it is enough to wait for several messages.



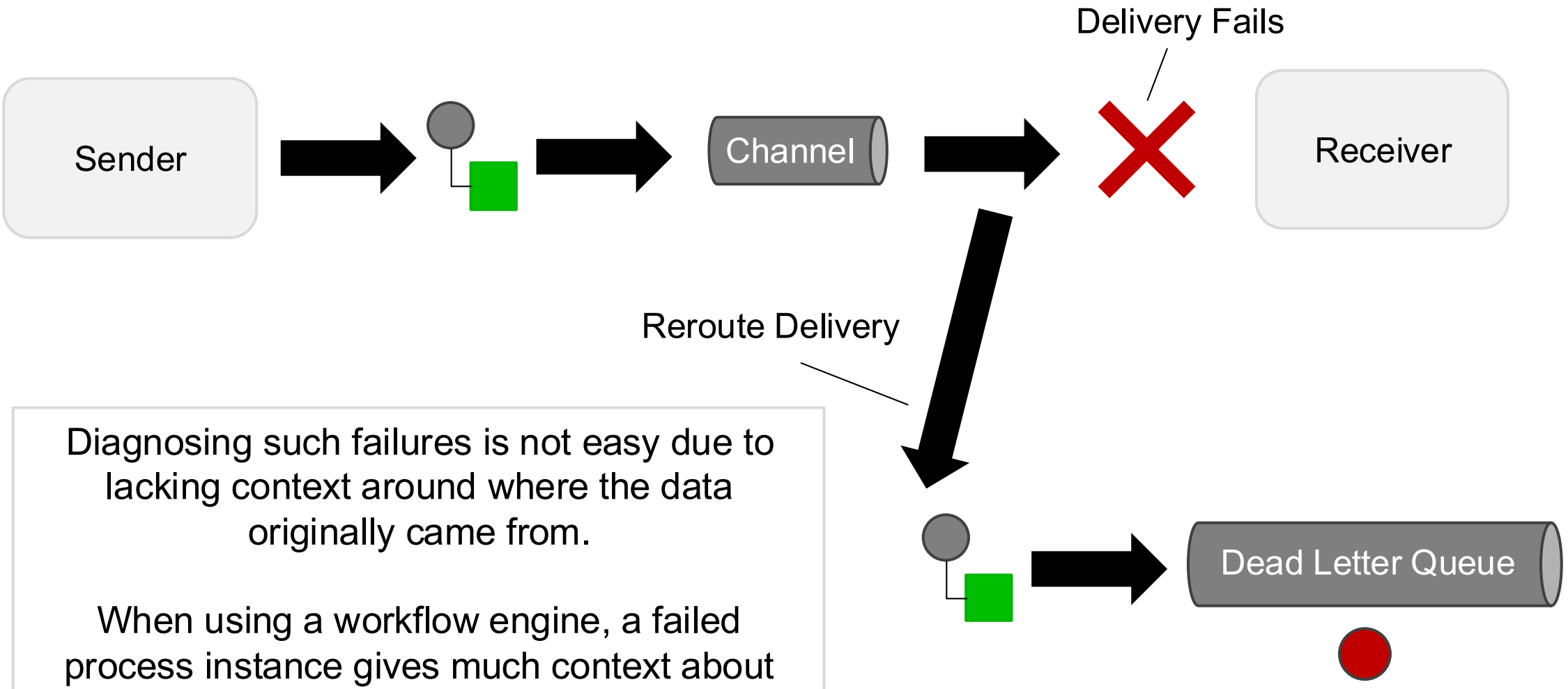
Poisoned and Dead Messages



The sender (e.g., a service / microservice) tries to send a persistent message through a channel to an intended receiver. The delivery to the receiver fails (e.g., the Receiver throws an exception). This leads to several retries and finally the message is moved to a Dead Letter Queue.

Source: Enterprise Integration Patterns

Poisoned and Dead Messages



Diagnosing such failures is not easy due to lacking context around where the data originally came from.

When using a workflow engine, a failed process instance gives much context about where it started, which path it took, and what data is attached.

Source: Enterprise Integration Patterns

Synchronous Facades Hiding Asynchronous Communication

Need for synchronous API for clients (e.g., frontends), while using asynchronous communication for long-running processes in your architecture

```
try {
   sendRequestToServiceB(correlationId, ...)
   response = waitForResponseFromServiceB(correlationId, timeout) //
   …
   }
catch (timeoutError) {
   // ?
}
```

- For receiving a response, you
  - subscribe to the channel that delivered the response message
  - provide a callback API
  - regularly poll

Timeouts play a role in all three cases. A workflow engine can help here.

Universität St.Gallen

Synchronous Facades Hiding Asynchronous Communication

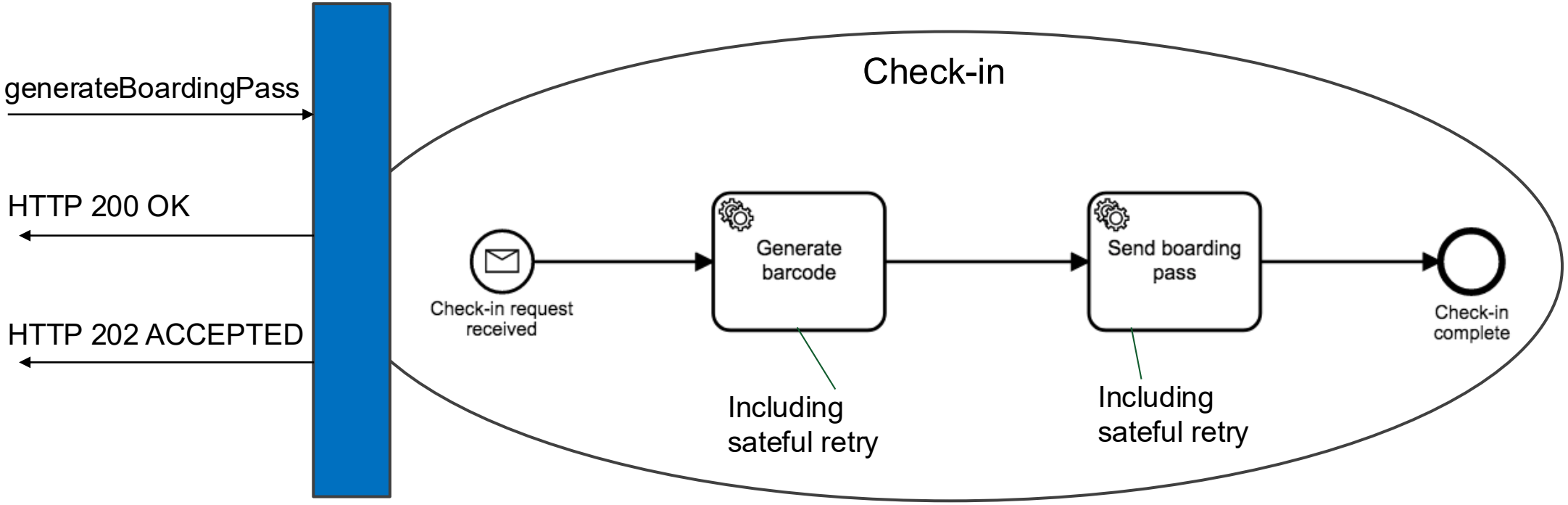
Need for synchronous API for clients (e.g., frontends), while using asynchronous communication for long-running processes in your architecture

- When responding asynchronously we need a way to deliver the result back to the caller.
  - Callback URL
  - Polling
  - Subscription

Timeouts play a role in all three cases. A common pattern is to *return synchronously* when everything is all right, and to *fall back to asynchronous* in case of an error.

Synchronous Best Effort

A Workflow Engine Does not Enforce Asynchronicity



In the happy case a synchronous response is made, otherwise it is switched to asynchronous processing.

Agenda

- Communication patterns for service invocation, in particular, long-running and asynchronous communication
- **Consistency problems and transactional guarantees**
- Importance of idempotence

ACID Transactions Ensure Consistency Within the Boundary

One joined ACID transaction

```
try {
   transaction.begin();
   createCustomer(..);
   createBillingData(..);
   transaction.commit();
} catch (Exception ex) {
   transaction.rollback();
}
```

Customer tables

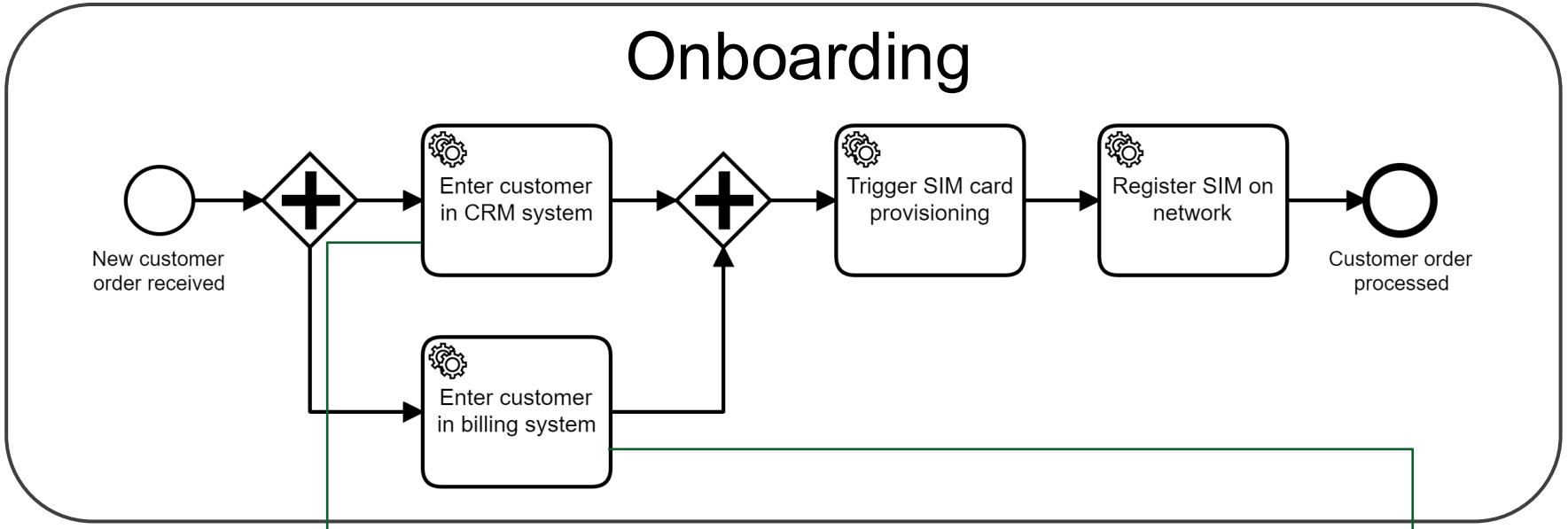
ID	Address	…

Billing tables

ID	Account	…

RDMS

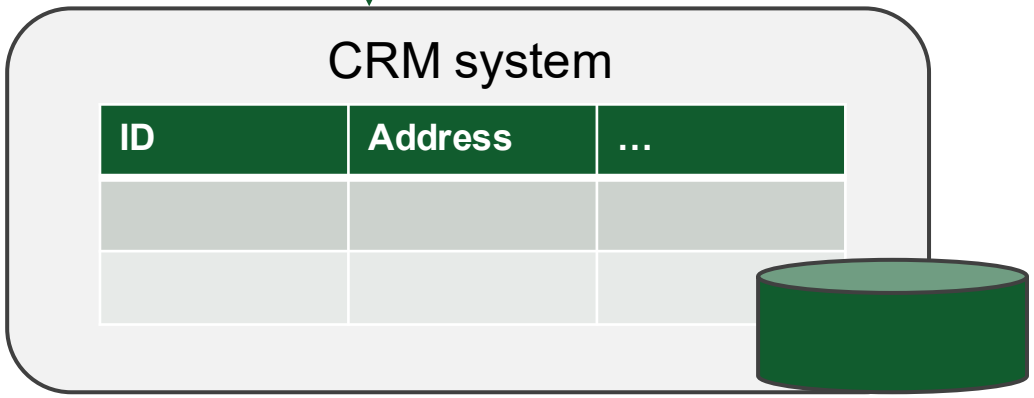
Each Service Has Its Own ACID Transactions

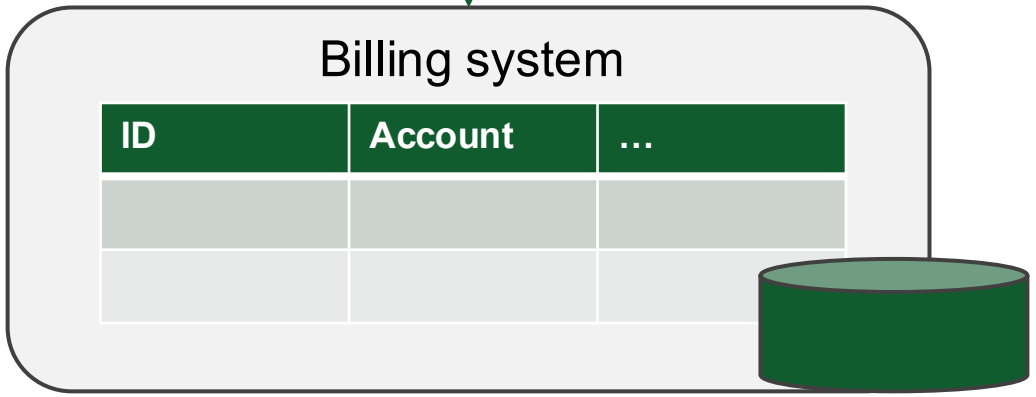


Changes made in intermediate steps will be immediately visible to the world

(violating **isolation**)

Each service has its own ACID transaction



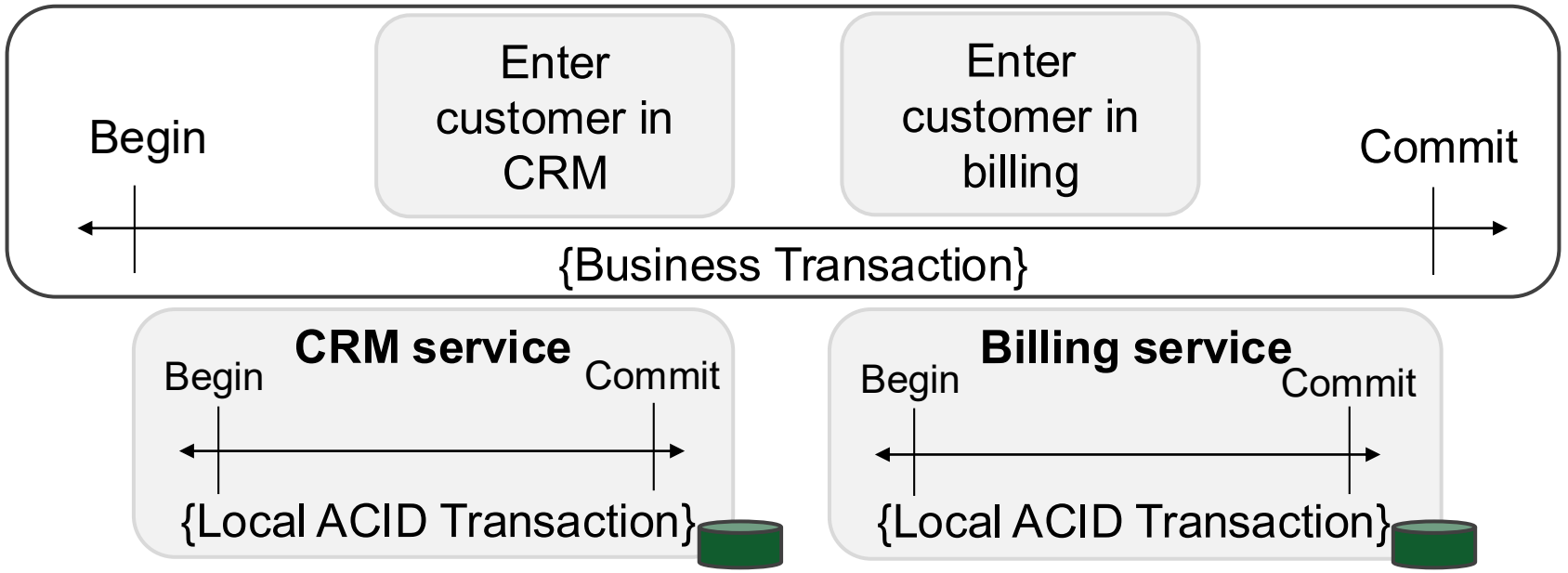


Challenges Around Atomicity

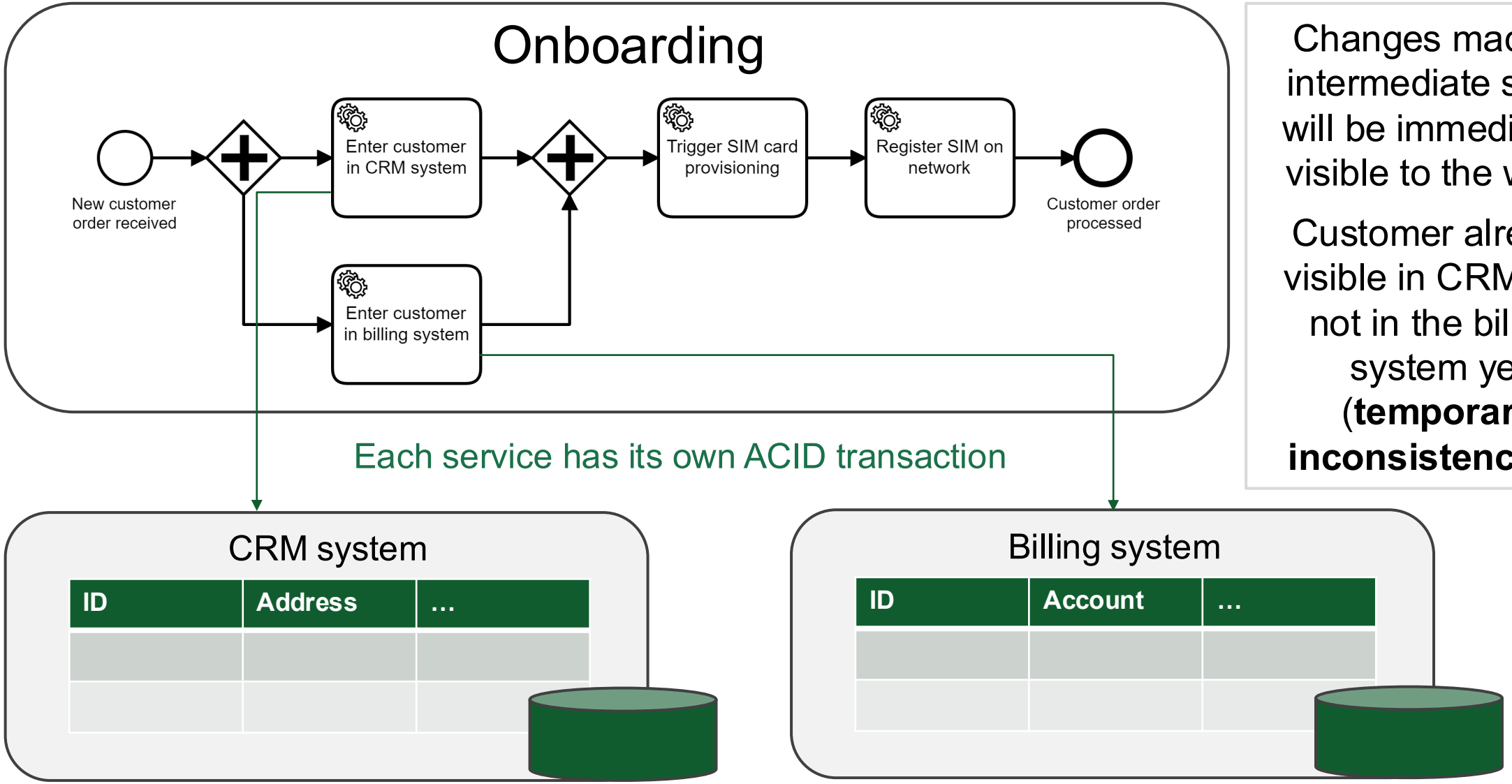
- Systems more and more distributed
  - Technologies for distributed transactions like the two-phase-commit protocol are expensive, complex, and brittle
  - Thus, normal projects should assume that ACID transactions are not possible in the presence of remote communication
- Different resources (e.g., multiple database installations or middleware) often cannot join a common ACID transaction
- Activities become long-running; Transactions cannot be held open (deadlocks, timeouts)
- Activities get to complex to be handled in one huge transaction

The Need for Business Transactions

- Business transactions cross boundaries, while ACID transaction can only happen within a boundary
- Need to loosen consistency expectations, since not all tasks are isolated during a business transaction
- Ensure that once started all tasks of a business transaction are either carried out or rolled back



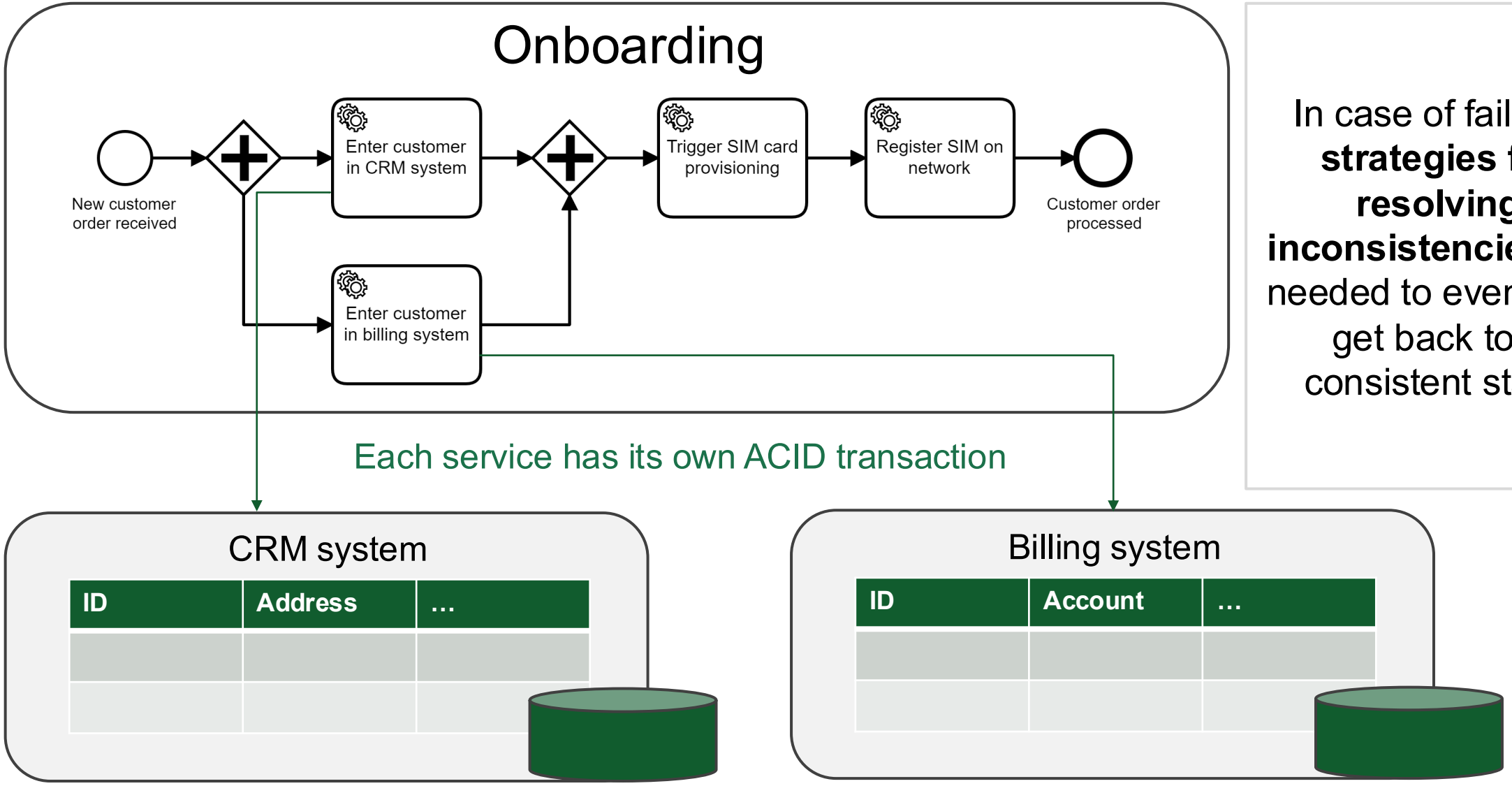
Eventual Consistency



Changes made in intermediate steps will be immediately visible to the world

Customer already visible in CRM, but not in the billing system yet (**temporary inconsistencies**).

Eventual Consistency

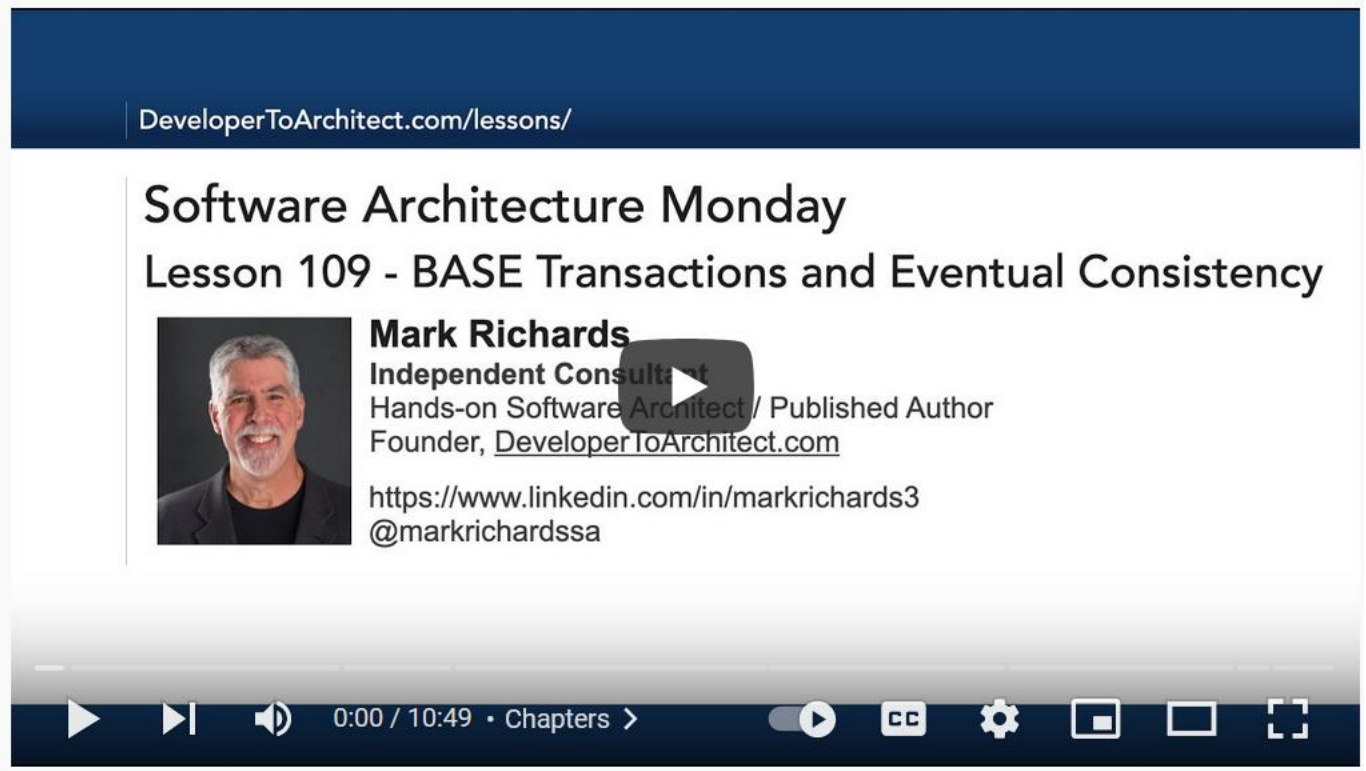


In case of failures **strategies for resolving inconsistencies** are needed to eventually get back to a consistent state.

Business Strategies to Handle Inconsistencies

- Ignore the Consistency
  - Example: In the context of the onboarding example, you might decide to ignore dead entries in the CRM because they occur rarely and have little business impact
- Apologize
  - Example: We might decide to ignore failures in the SIM registration, wait for the customers to complain, send an apology with a $10 voucher, and trigger the registration manually
- Resolve the Consistency
  - Example: Through reconciliation jobs (typically run as a batch job), Saga pattern, outbox pattern

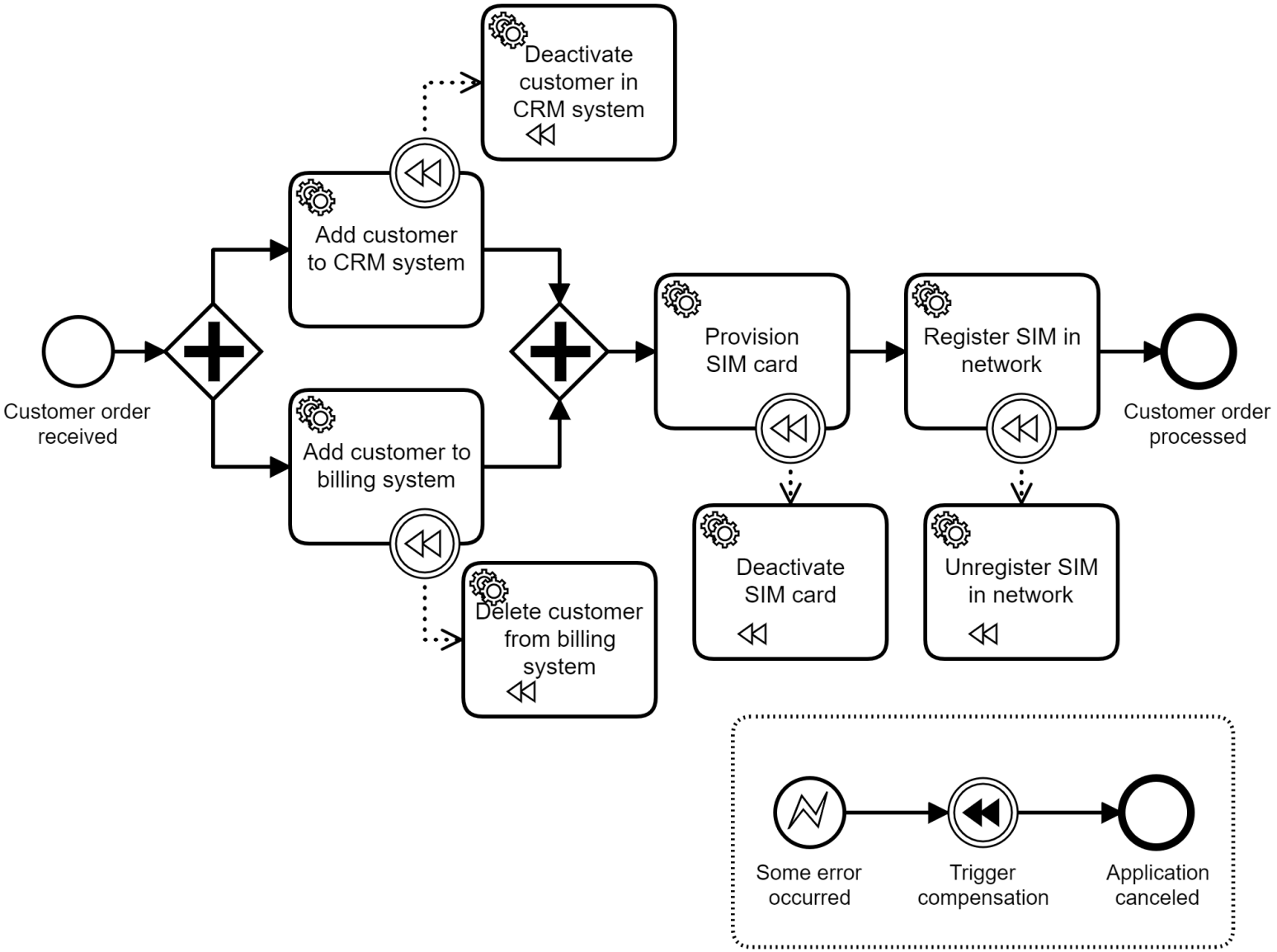
Resolving Inconsistencies



- Background synchronization
- Request-based synchronization
- Event-based synchronization

Further details in book „Software Architecture: The Hard Parts“, Chapter 7

Saga Pattern

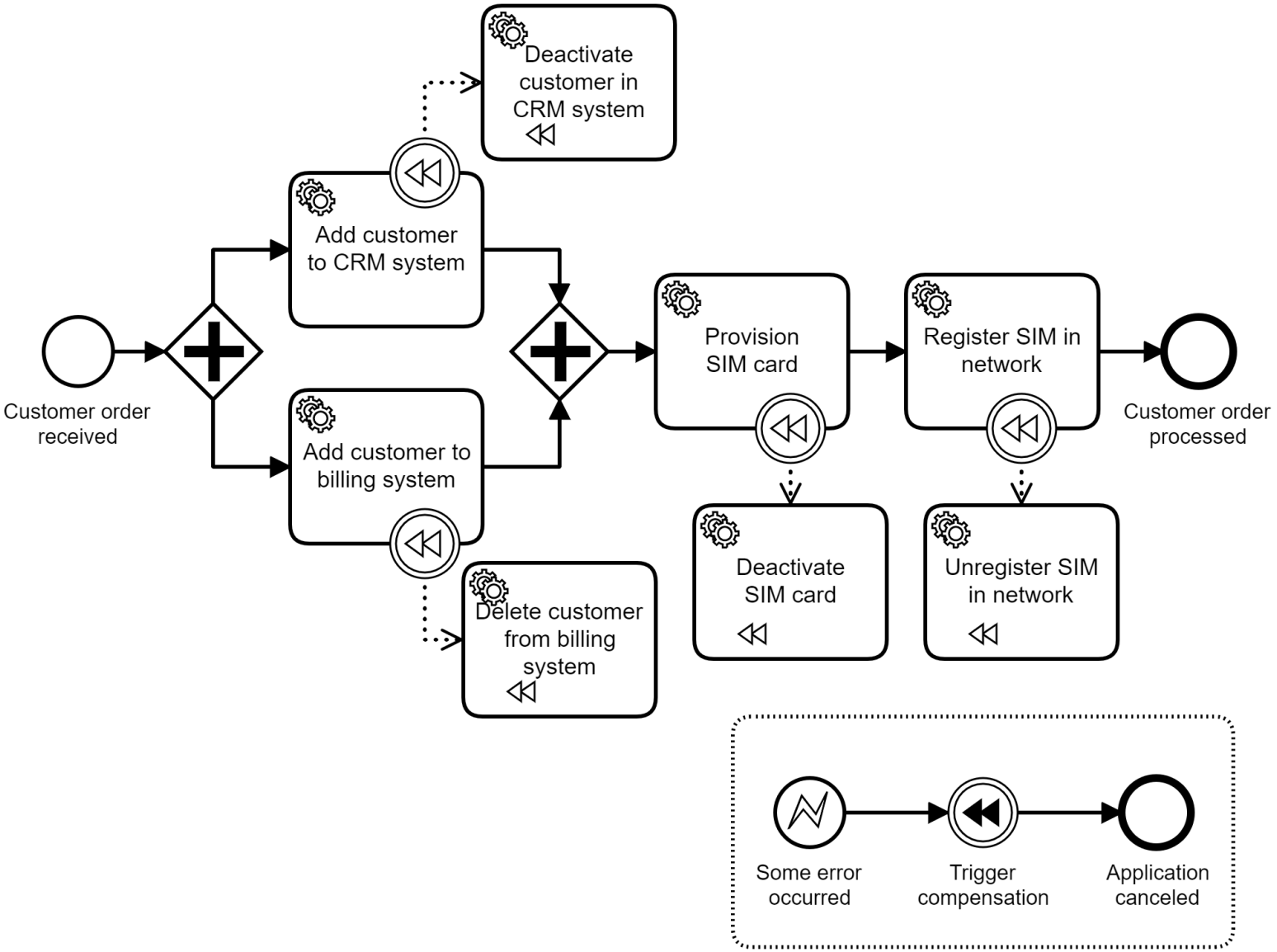


A saga models a business transaction across multiple services.

It is implemented as a sequence of local transactions.

If a step fails, previously completed steps are compensated using explicit undo actions.

Saga Pattern



In distributed systems, ACID transactions cannot span multiple services.

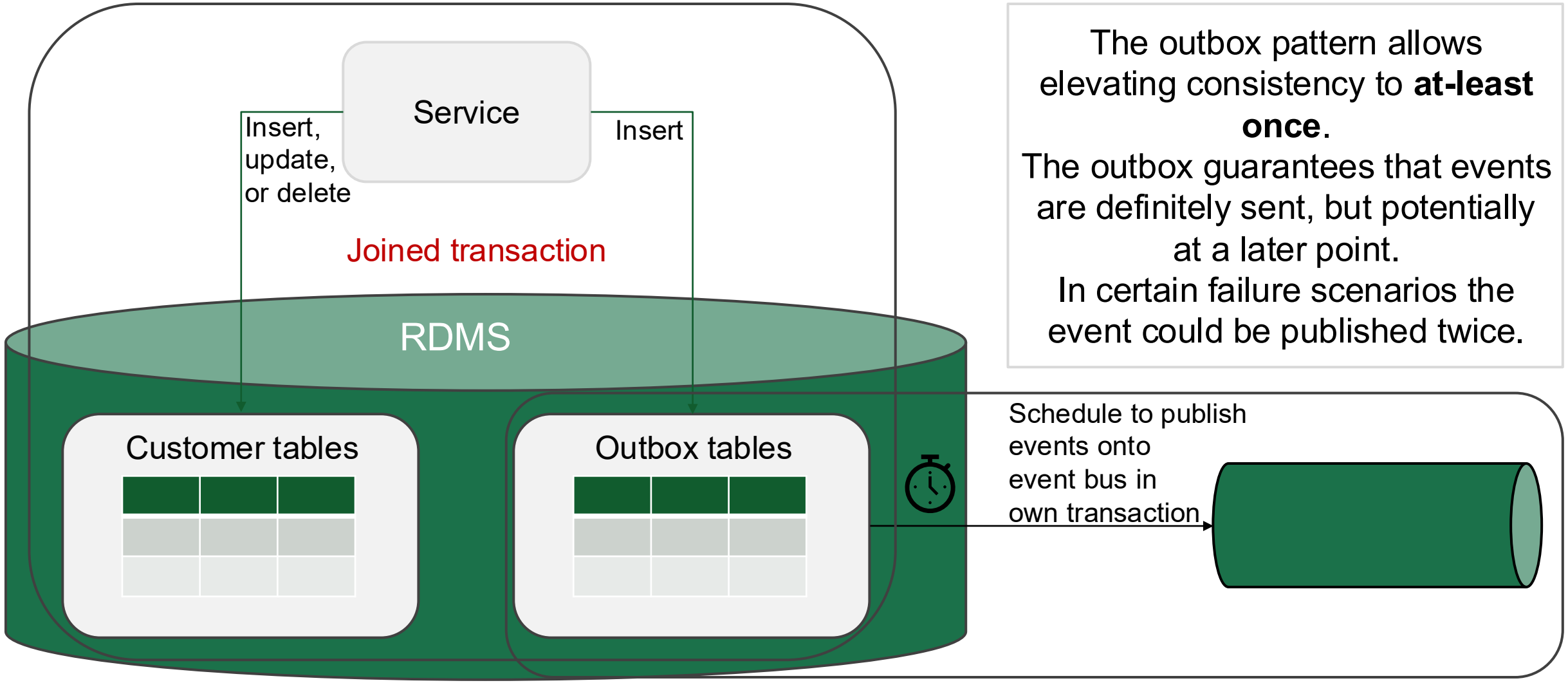
As a result, transaction management moves to the application layer.

Sagas provide a structured way to coordinate local transactions and compensating actions to achieve eventual consistency.

Distributed Sagas – An Overview

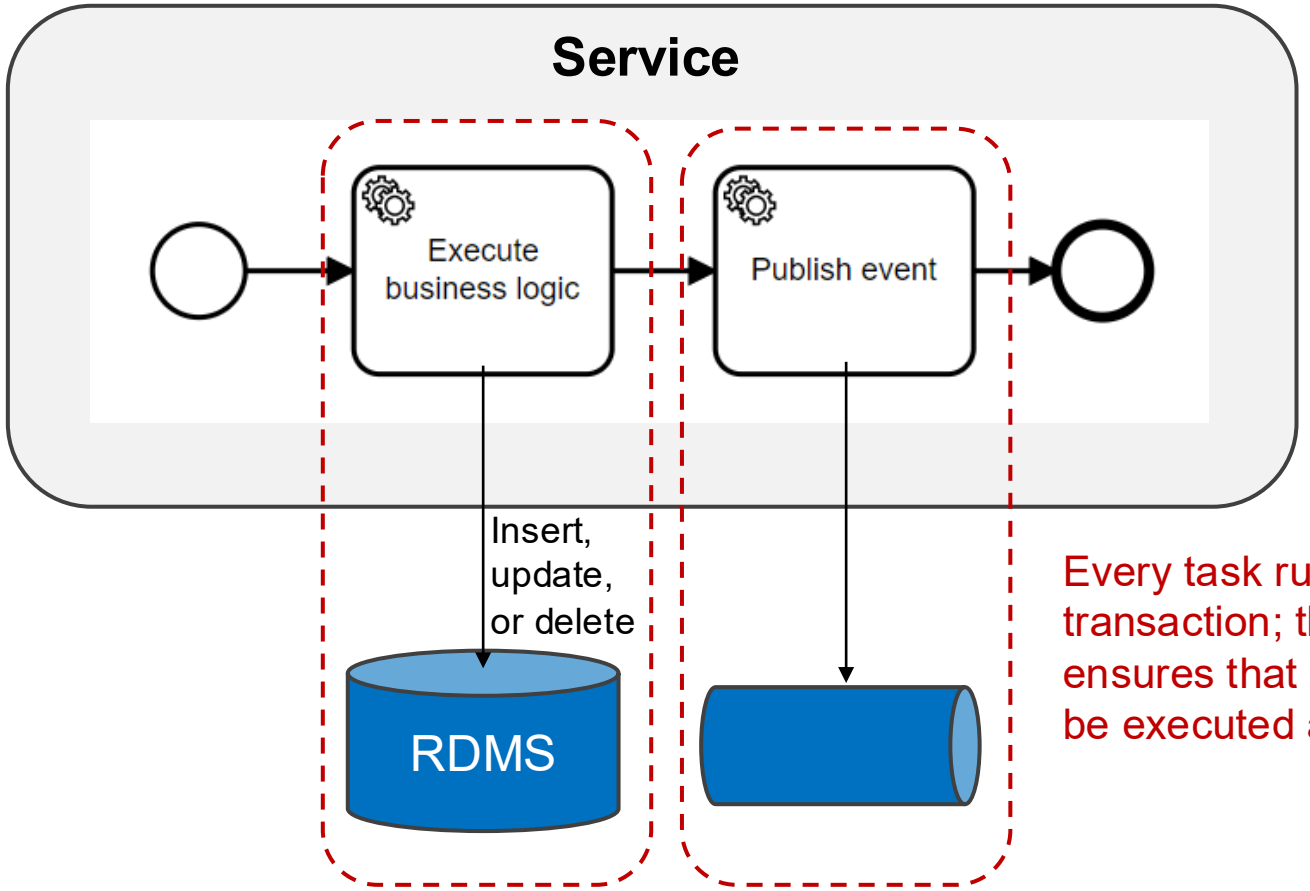
Source: https://youtu.be/0UTOLRTwOX0

Chaining Resources Using the Outbox Pattern



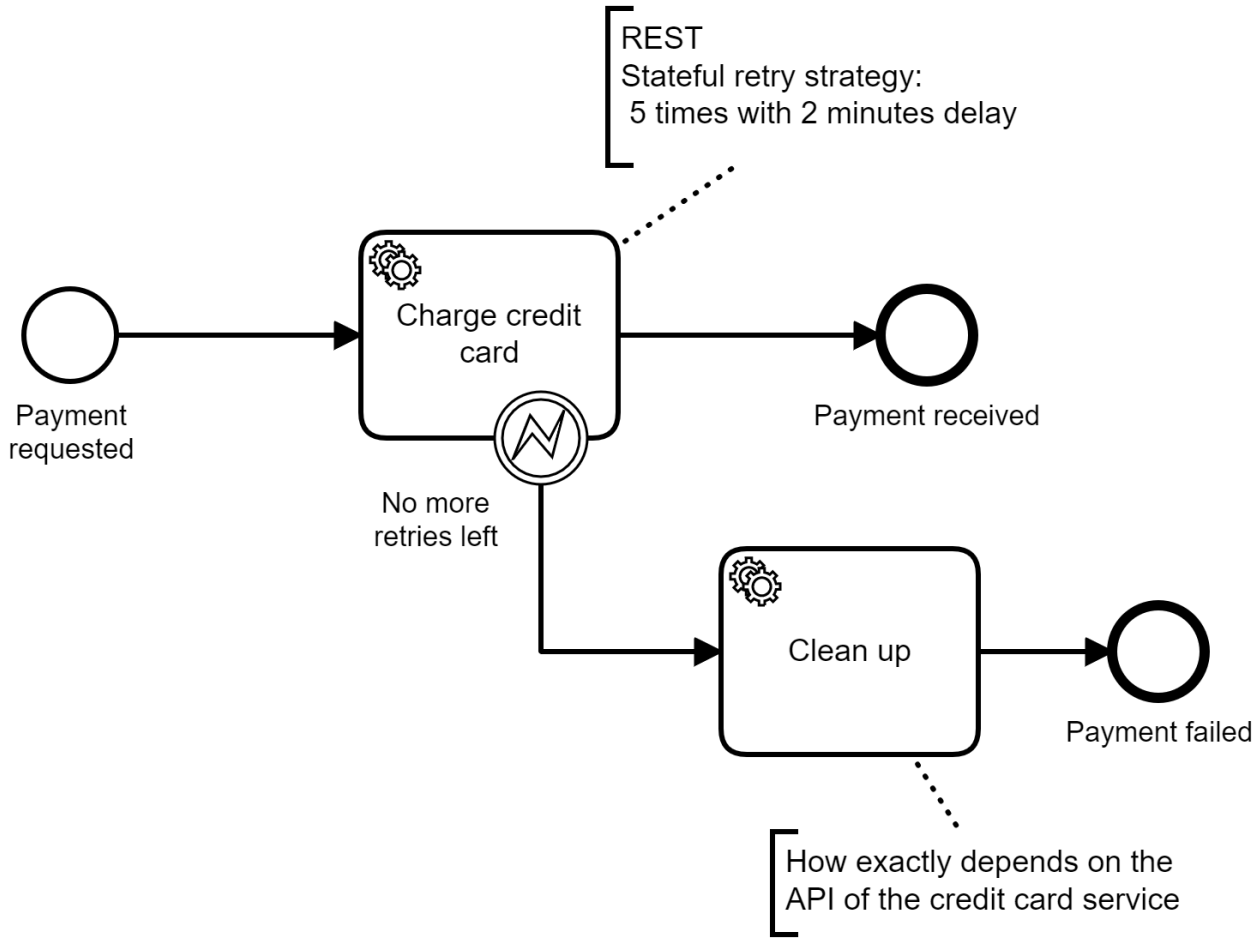
The outbox pattern allows elevating consistency to **at-least once**.
The outbox guarantees that events are definitely sent, but potentially at a later point.
In certain failure scenarios the event could be published twice.

Implementing the Outbox Pattern with a Workflow Engine



Every task runs a seperate transaction; the workflow engine ensures that all tasks will be executed at least once

Eventual Consistency Applies to Every Form of Communication



Imagine the payment service needs to call the charge credit card service via a REST API.

Assume this call yields a network exception.

Has the credit card been charged or not?

Agenda

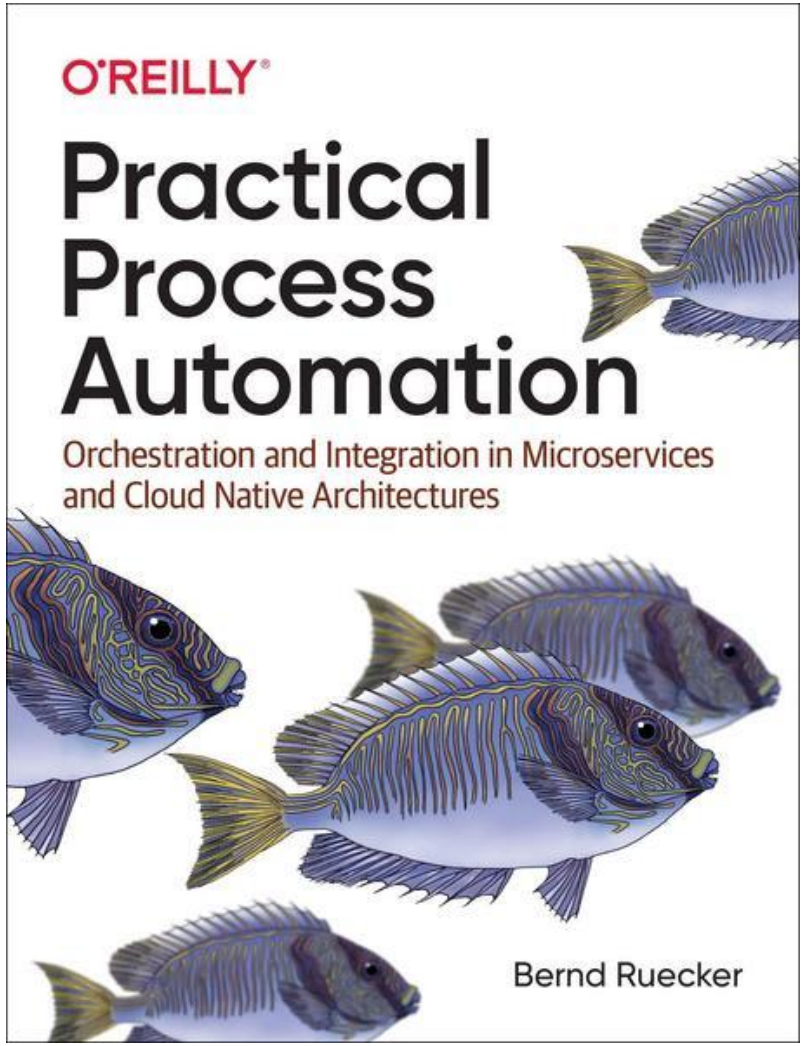
- Communication patterns for service invocation, in particular, long-running and asynchronous communication
- Consistency problems and transactional guarantees
- **Importance of idempotence**

The Importance of Idempotence

Idempotence: Operations that can be applied multiple times without changing the result beyond the initial application

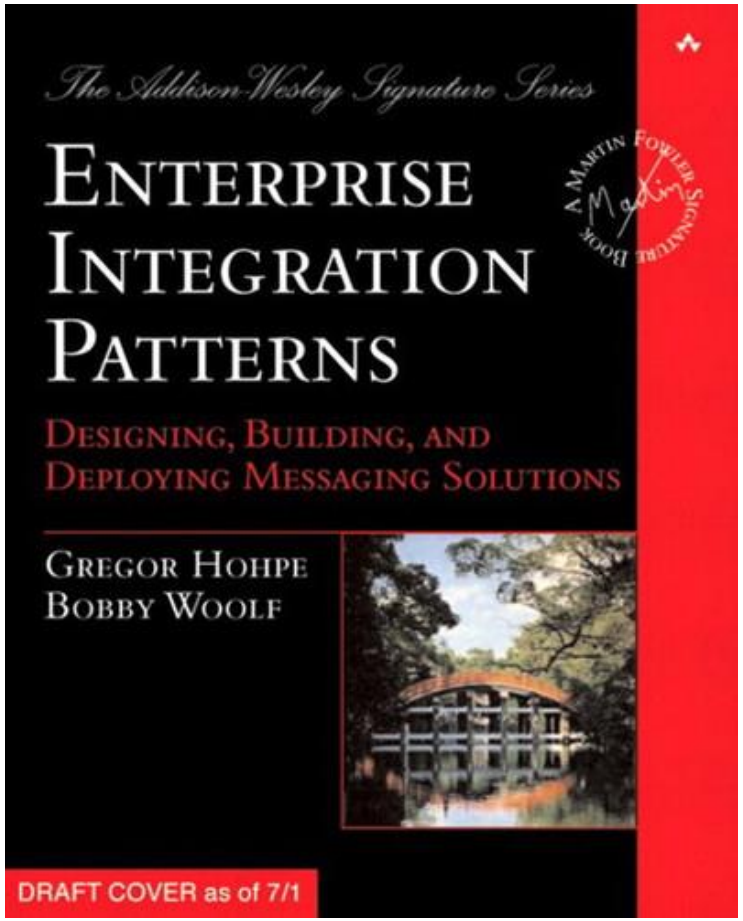
- Repeated calls are unproblematic for idempotent operations
  - Retries or redelivery of messages
- Idempotent operations
  - Queries
  - Deletions (not possible to delete the same entity twice; response might differ)

Further References



Based on Chapter 9 – Workflow Engines and Integration Challenges

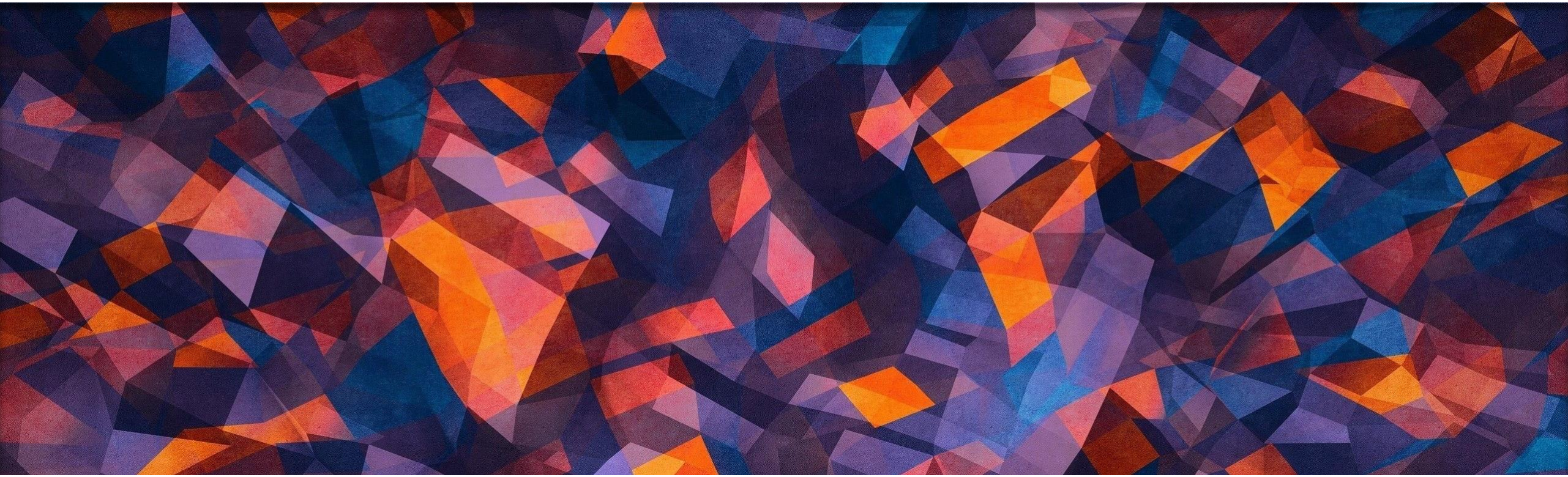
Further References



Further References

- https://blog.bernd-ruecker.com/fail-fast-is-not-enough-84645d6864d3
- https://camunda.com/blog/2018/08/stateful-orchestration-handle-errors-responsibly/
- https://blog.bernd-ruecker.com/service-integration-patterns-with-bpmn-and-camunda-cloud-53b0f458e49
- BPMN and Microservices Orchestration, Part 1 of 2: Flow Languages, Engines, and Timeless Patterns
- BPMN and Microservices Orchestration, Part 2 of 2: Graphical Models, Simplified Sagas, and Cross-functional Collaboration
- https://blog.bernd-ruecker.com/saga-how-to-implement-complex-business-transactions-without-two-phase-commit-e00aa41a1b1b
- https://microservices.io/patterns/data/saga.html

Universität St.Gallen



Event-driven and Process-oriented Architectures

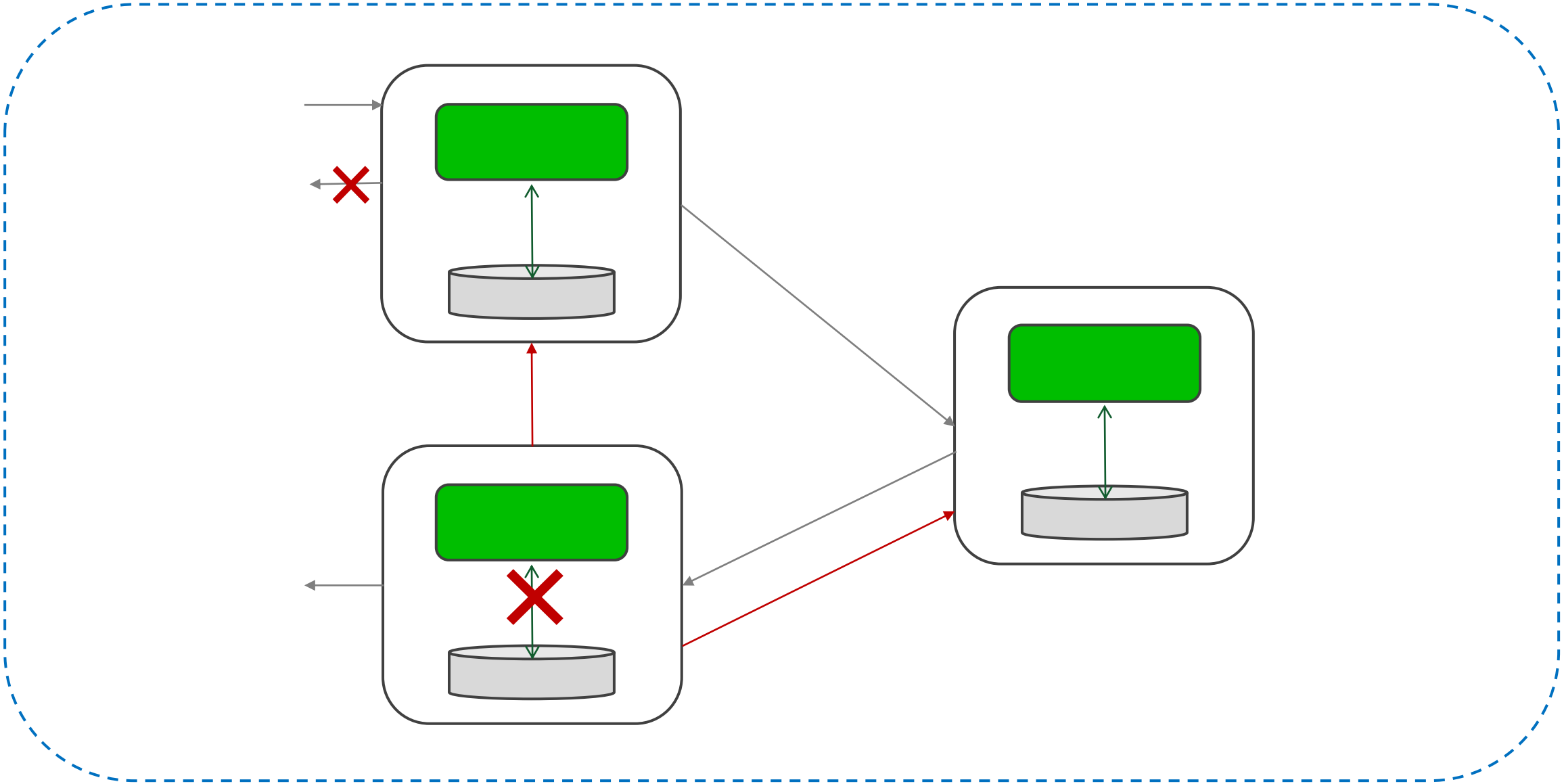
Appendix – Additional Saga Variants

Lecture 5

Phone Tag Saga

- Due to the lack of orchestration, each participant must coordinate status
- The initially called service becomes the coordination point (sometimes called *front controller*)
- In case an error condition occurs, each service requires built-in logic to send compensating requests back along the chain
- Typically used for simple workflows, that need higher scale, but with a potential performance impact

Phone Tag Saga



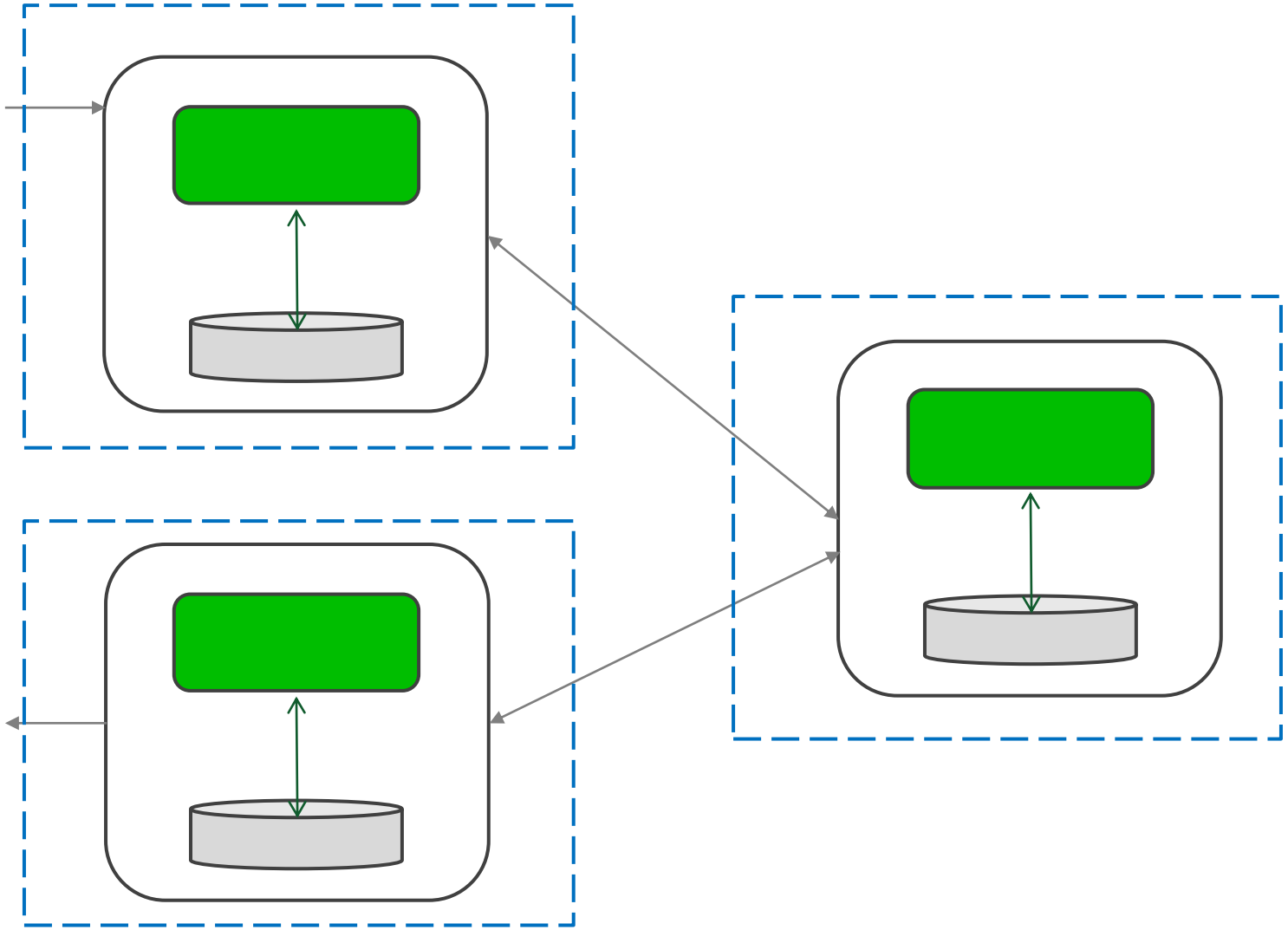
Phone Tag Saga

Attribute	Rating
Communication	Synchronous
Consistency	Atomic
Coordination	Choreographed
Coupling	High
Complexity	High
Responsiveness/Availability	Low
Scalability/Elacticity	Low

Time Travel Saga

- Avoids a central mediator and places the workflow responsibilities entirely on the participating domain services
- This architecture can implement any workflow with any one-way series of steps (Chain of Responsibility design pattern or Pipes and Filters architectural style)
- Each service owns its own transactionality, making workflow consistency temporarily gradual
- For solutions benefiting from high throughput, this pattern works well for “fire and forget workflows” such as electronic data ingestion or bulk transactions

Time Travel Saga



Time Travel Saga

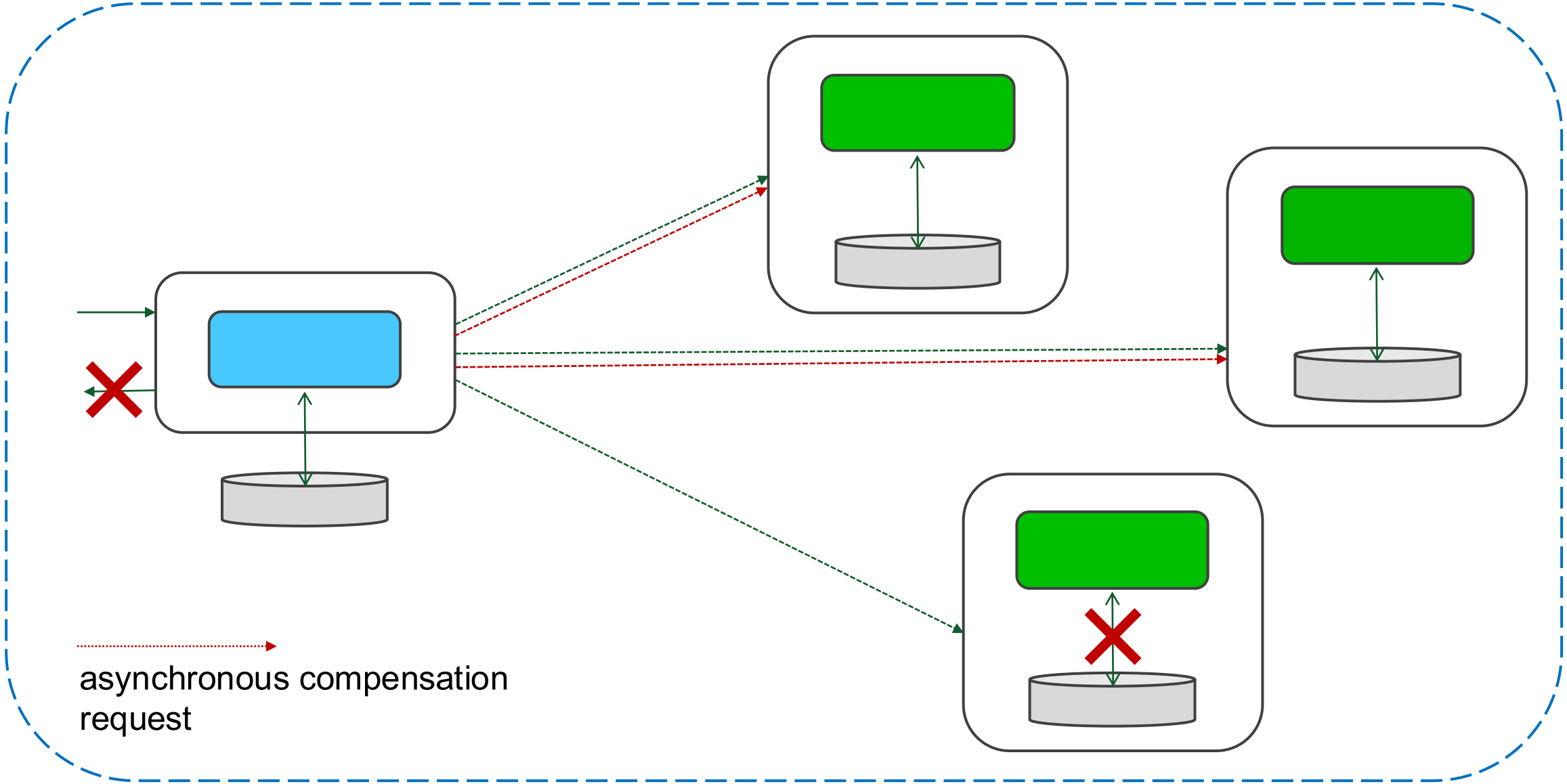
Attribute	Rating
Communication	Synchronous
Consistency	Eventual
Coordination	Choreographed
Coupling	Medium
Complexity	Low
Responsiveness/Availability	Medium
Scalability/Elacticity	High

Due to synchronous communication simpler than the Anthology Saga pattern. If scalability sufficient, architects can embrace this simpler pattern.

Fantasy Fiction Saga

- This pattern combines asynchronous communication, with atomic consistency and orchestration and apart from the asynchronous communication resembles the Epic Saga pattern
- It is a relatively implausible combination, which is used more frequently than it should be (Parallel Saga often the better option)
- Improves preformance when compared to the Epic Saga, but adds asynchronous transactional state (and thus many challenges)

Fantasy Fiction Saga



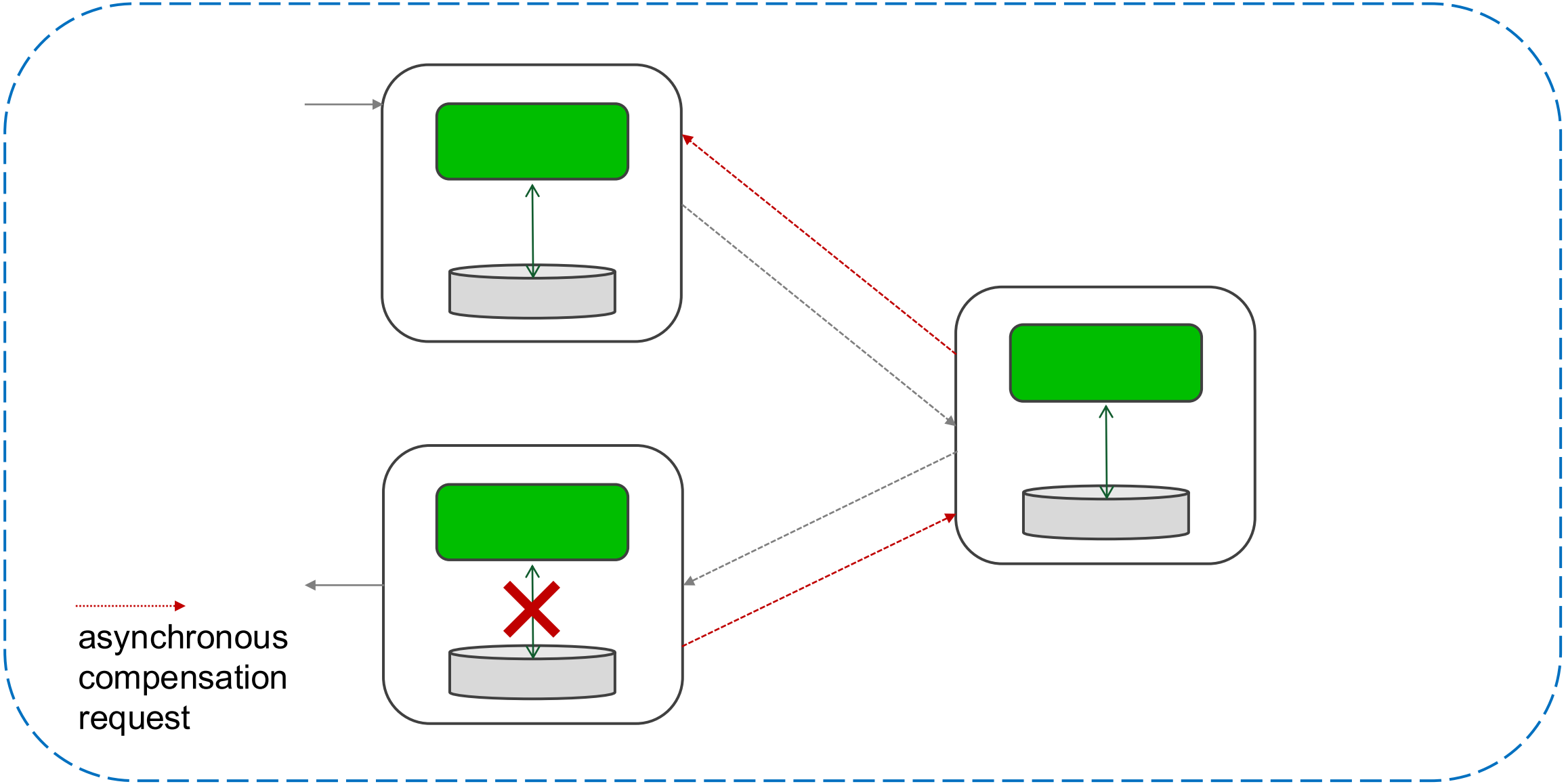
Fantasy Fiction Saga

Attribute	Rating
Communication	Asynchronous
Consistency	Atomic
Coordination	Orchestrated
Coupling	High
Complexity	High
Responsiveness/Availability	Low
Scalability/Elacticity	Low

Horror Story Saga

- This pattern combines the most stringent coupling around consistency with the two lowest coupling styles (asynchronous and choreography)
- Lack of a mediator to manage transactional consistency across multiple services
- Each domain service must track undo information around multiple pending transactions, possibly out of order (due to asyncrhonisity), and coordinate with each other due to error conditions
- Anthology Saga might often be the better option

Horror Story Saga



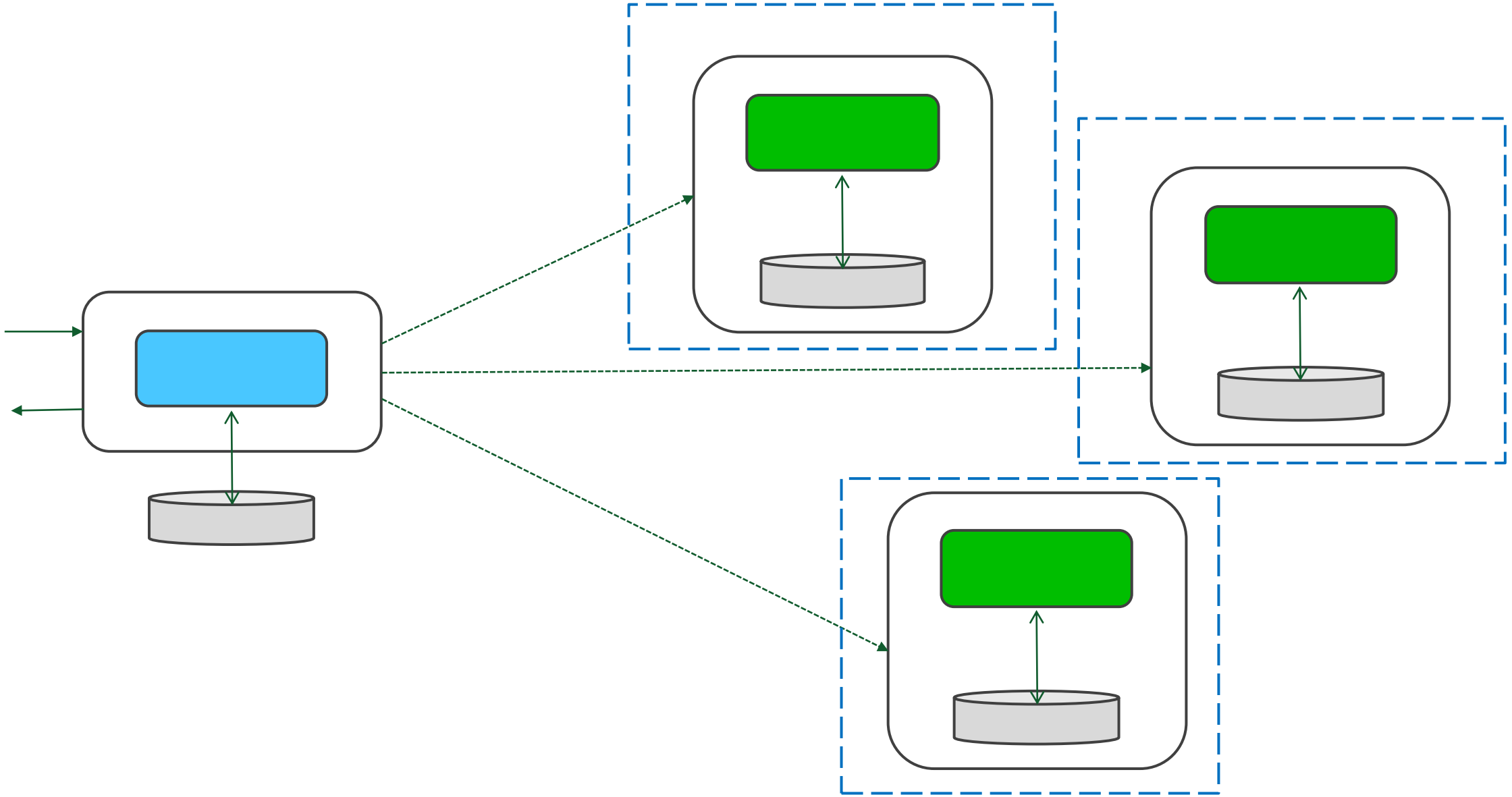
Horror Story Saga

Attribute	Rating
Communication	Asynchronous
Consistency	Atomic
Coordination	Choreographed
Coupling	Medium
Complexity	Very high
Responsiveness/Availability	Low
Scalability/Elacticity	Medium

Parallel Saga

- Two key differences when compared to the traditional saga (asynchronous communication and eventual consistency) make it easier to implement
- Uses a mediator and is therefore suitable for complex workflows
- Asynchronous communication allows for better responsiveness and parallel execution
- Consistency lies with the domain services (e.g., to synchronize shared data)

Parallel Saga

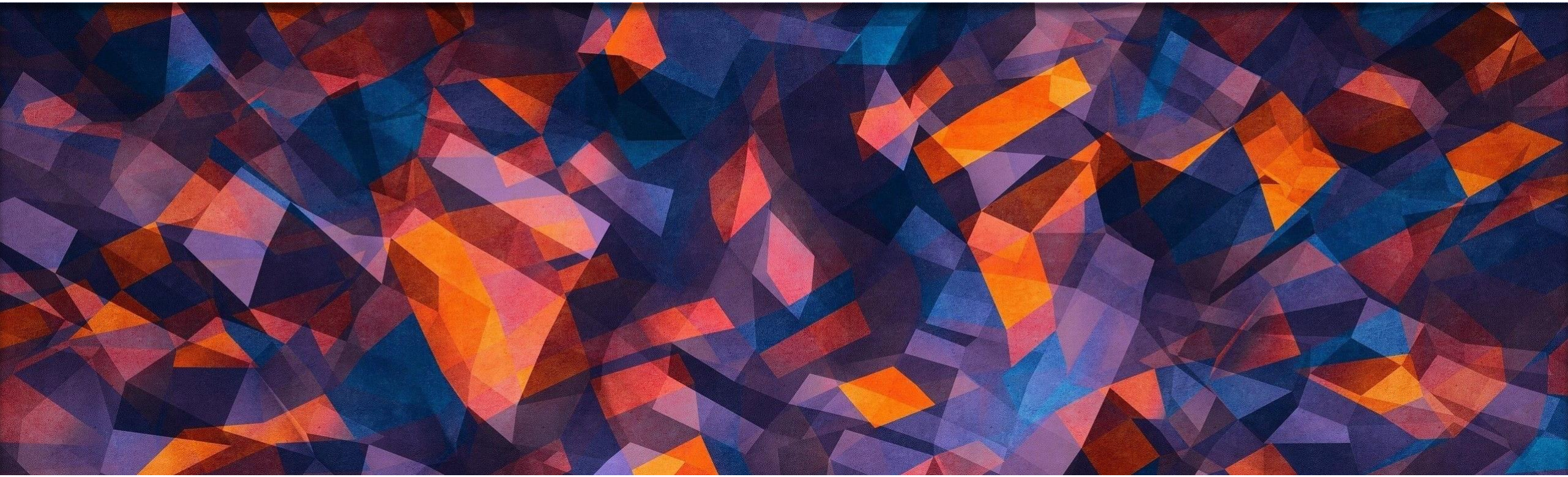


Parallel Saga

Attribute	Rating
Communication	Asynchronous
Consistency	Eventual
Coordination	Orchestrated
Coupling	Low
Complexity	Low
Responsiveness/Availability	High
Scalability/Elacticity	High

Attractive trade-off for many scenarios, in particular with complex workflows and the need for high scale.

Universität St.Gallen



Event-driven and Process-oriented Architectures

Exercise

Lecture 5

Labs Week 5

- Lab 8: Stateful resilience patterns
  - https://github.com/scs-edpo/lab04-flowing-retail/tree/master/rest

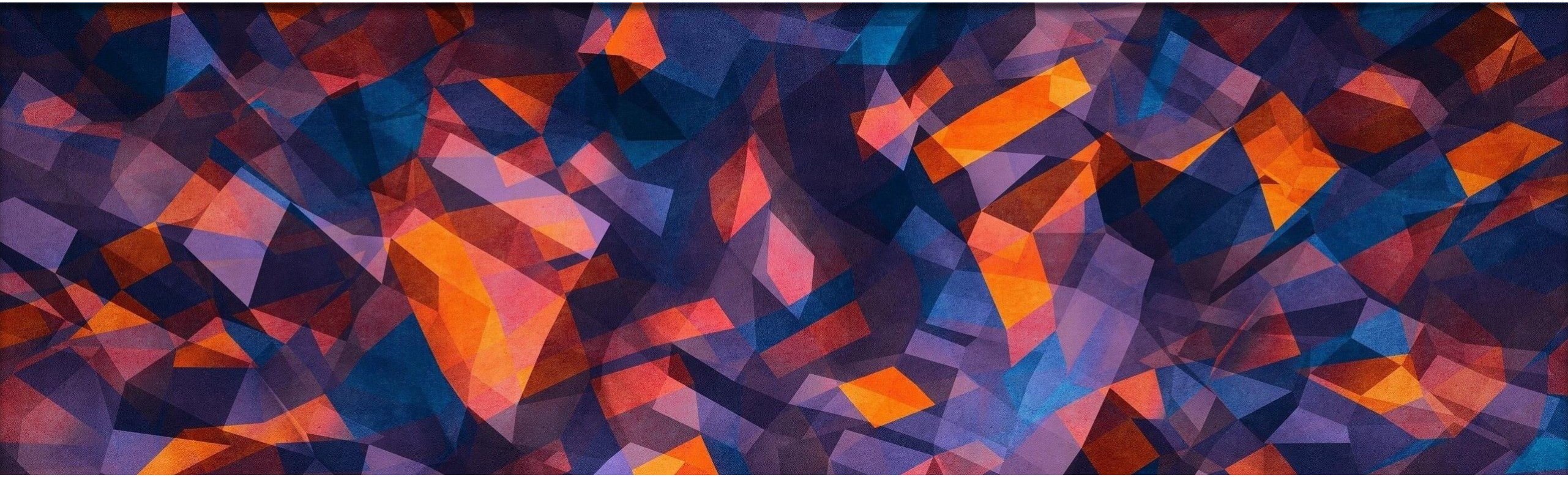
- Blog articles:
  - https://blog.bernd-ruecker.com/fail-fast-is-not-enough-84645d6864d3
  - https://camunda.com/blog/2018/08/stateful-orchestration-handle-errors-responsibly/

Labs Week 5

- Lab 9: Trip Booking Saga
  - https://github.com/scs-edpo/lab09-trip-booking-saga

- Blog Article:
  - https://blog.bernd-ruecker.com/saga-how-to-implement-complex-business-transactions-without-two-phase-commit-e00aa41a1b1b

Universität St.Gallen



Event-driven and Process-oriented Architectures

Ideas for Project

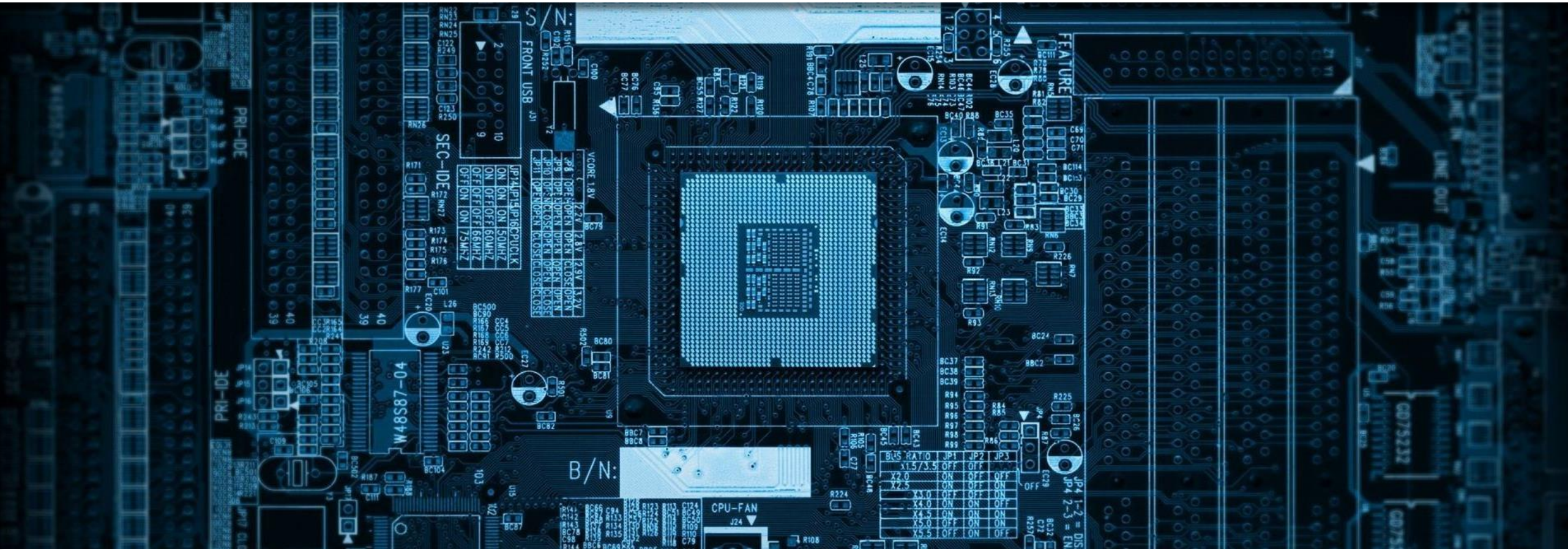
Lecture 5

Week 5

- Implement one or several of the following stateful resilience patterns
  - Stateful retry
  - Human intervention
  - Outbox and/or Saga patterns

  Hint: There is no need for including transactions. It is ok to implement transactional sagas based on state management and eventual consistency.

  Following the talk by C. McCaffrey you need for Sagas:
  - **Durable Saga log** — this is implemented by the workflow engine
  - **SEC Process** (Execution Coordinator) — this is implemented by the workflow engine
  - **Idempotence of compensating actions** — this is up to you in this scenario as it has to be implemented by the services you call in your Saga (e.g., cancelling the car booking).



Questions?